

INTERNSHIP REPORT

*A report submitted in partial fulfillment of the requirements for the Award of Degree
of*

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

By

AKSHAT GARG

Regd. No.: 23BCE10641

Under Supervision of

MPOnline Limited, Bhopal

(AI/ML Internship Program)

(Duration: 20th May, 2026 to 31st July, 2026)



VIT[®]
B H O P A L
www.vitbhopal.ac.in

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING SCIENCE AND ENGINEERING (SCOPE)**

VIT BHOPAL UNIVERSITY

2023 – 2027

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
VIT BHOPAL UNIVERSITY



CERTIFICATE

This is to certify that the "**Internship Report**" submitted by **AKSHAT GARG** (Regd. No.: **23BCE10641**) is the work done by the student during the **AI/ML Internship Program at MPOnline Limited, Bhopal**, from **20th May, 2026 to 31st July, 2026**, and is submitted in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING (2023 – 2027)** at VIT Bhopal University.

College Internship Guide

Signature: _____

Manikandan Jayapal

Associate Professor

VIT Bhopal University

CERTIFICATION



Certificate of Completion issued by MPOonline Limited for the Internship in AI/ML (Certificate ID: MPO/INT/26-27/07/359/26050363; Duration: 20th May, 2026 to 31st July, 2026).

ACKNOWLEDGEMENT

First, I would like to thank the management of **MPOnline Limited, Bhopal**, and **Gagan Soni, Chief Technical Officer, MPOnline Limited**, for giving me the opportunity to carry out my internship in Artificial Intelligence and Machine Learning within the organisation.

I am deeply grateful to my instructor, **Nishant Shrivastava**, who conducted the sessions of the AI/ML Internship Program (Batch 1A). The clear explanations, carefully prepared notes and assignments, and prompt guidance whenever I was stuck made it possible for me to learn a very wide range of topics in a short time and to build real, deployed applications.

I would also like to thank my fellow participants of Batch 1A. Their questions and discussions during the sessions created an open and enjoyable learning environment.

It is indeed with a great sense of pleasure and an immense sense of gratitude that I acknowledge the help of the following individuals at VIT Bhopal University.

I am highly indebted to **Dr. Pushpinder Singh Patheja, Professor and Dean, School of Computing Science and Engineering (SCOPE)**, for the facilities and encouragement provided to accomplish this internship.

I would like to thank **Dr. Ravi Verma, Program Chair, CSE-Core, SCOPE**, for the support and constructive suggestions provided throughout my internship.

I would like to thank my College Internship Guide, **Manikandan Jayapal, Associate Professor**, for the support and advice given to me in completing the internship and in preparing this report.

I am extremely grateful to the staff members of my department, my friends and my family, who helped and encouraged me in the successful completion of this internship.

AKSHAT GARG
(23BCE10641)

EXECUTIVE SUMMARY / ABSTRACT

1. The Company

MPOnline Limited is the e-governance service delivery company of the Government of Madhya Pradesh. It was incorporated in 2006 as a joint venture between the Government of Madhya Pradesh and Tata Consultancy Services (TCS) and is based in Bhopal. MPOne builds and operates the state's citizen services portal, www.mponline.gov.in, through which government departments deliver citizen, business and government services such as online applications, examinations and recruitment, counselling, licences and certificates, supported by a statewide network of authorised kiosks. The organisation is ISO certified and CMMI Maturity Level 3 appraised. It also conducts technology internships for students; I completed its AI/ML Internship Program (program code AI3, Batch 1A) from 20th May, 2026 to 31st July, 2026.

2. The Problem or Opportunity

Organisations such as MPOne process very large volumes of structured data, including applications, examination records and payments, and machine learning offers ways to use this data: forecasting demand, detecting unusual transactions, classifying requests and answering citizen queries automatically. Using these techniques well requires more than training a model. The model must be evaluated honestly, packaged as a service and deployed where real users can reach it. The opportunity addressed by this internship was to build that complete, end-to-end competence, bridging the gap between classroom theory and the way machine learning systems are actually built, versioned and deployed in industry.

3. Methodology

The program ran from Monday to Friday, 1:00 pm to 3:00 pm, under instructor Nishant Shrivastava. Each module combined a concept session supported by detailed notes, guided coding in Python with Jupyter Notebook and VS Code, and a practical exercise or project. The curriculum progressed from Python, NumPy and Pandas, through mathematics for ML, exploratory data analysis and feature engineering, to supervised learning (regression, KNN, decision trees, random forests and SVM), unsupervised learning (K-Means and PCA), deep learning with TensorFlow/Keras, NLP, prompt engineering and Retrieval-Augmented Generation, recommendation systems,

reinforcement learning with Stable-Baselines3, and finally deployment with Flask, Docker, GitHub and Render. All work was version-controlled in public GitHub repositories and submitted through the program's submission form.

4. Key Parts of the Report, Findings and Solutions

The report records the internship week by week, introduces the organisation and the program (Chapter 1) and discusses the work in detail (Chapter 2). The main project, a Car Price Prediction System, predicts used-vehicle resale prices with a Random Forest model (R^2 of 0.958 on unseen data) and is served through a Flask web application deployed live on Render and packaged with Docker. Chapter 2 also covers ten graded assignments and further projects, including a heart disease REST API, CNN image classifiers, RAG chatbots built with Google Gemini, and PPO agents for CartPole and Lunar Lander. An important finding was that a high score does not always mean a good model: reviewing the work revealed duplicate records that inflated one model's accuracy to 100%, a feature that leaked the answer into another model, and test sets that were reused for model selection. Each case is documented with its cause and the corrected approach. Annexure A summarises the Oracle Certified Foundations Associate (Java) course completed through PAT.

5. Benefits to the Company / Institution

For MPOnline, the internship produced a set of documented, reproducible and publicly hosted machine learning projects that can serve as reference implementations for future batches, together with a candid record of common pitfalls (data leakage, duplicate records, class imbalance and overfitting) and how to detect them. The deployed services show that ML models can be delivered on low-cost cloud infrastructure, a pattern directly relevant to e-governance uses such as service-demand forecasting and document question-answering for help desks. For VIT Bhopal University, the report documents industry-oriented training that complements the B.Tech CSE curriculum and brings back practical skills in deployment and rigorous model evaluation.

INDEX

S. No.	Contents	Page No.
	Certificate	ii
	Certification (Internship Certificate)	iii
	Acknowledgement	iv
	Executive Summary / Abstract	v
	Learning Objectives / Internship Objectives	ix
	Weekly Overview of Internship Activities	x
1	Introduction	1
1.1	Artificial Intelligence and Machine Learning: An Overview	1
1.2	About the Organisation: MPOne Limited	1
1.3	About the AI/ML Internship Program	2
1.4	Tools and Technologies Used	3
1.5	Organisation of the Report	3
2	Internship Discussion	4
2.1	How the Internship Objectives Were Achieved	4
2.2	Technical Learning During the Internship	5
2.3	Main Project: Car Price Prediction System	11
2.3.1	Problem Statement	11
2.3.2	Objectives of the Project	11
2.3.3	Software and Hardware Requirements	12
2.3.4	Dataset Description	12
2.3.5	Data Preprocessing and Feature Engineering	13
2.3.6	Model Development: Random Forest Regressor	14
2.3.7	Model Evaluation	15
2.3.8	System Architecture	16
2.3.9	Web Application using Flask	17
2.3.10	Deployment on Render	18
2.3.11	Containerised Version using Docker	20
2.3.12	Source Code	21
2.3.13	Output	24
2.3.14	Limitations and Future Enhancements	26
2.4	Graded Assignments	27
2.5	Additional Projects	39

S. No.	Contents	Page No.
2.6	Results and Observations	46
2.7	Skills Learned	49
2.8	Challenges Faced and How They Were Overcome	50
2.9	Project Repositories and Live Links	52
3	Conclusion	53
4	Bibliography	54
4.1	Books and Research Papers	54
4.2	Web References, Documentation and Data Sets	55
A	Annexure A: Summary of the PAT Course (Oracle Certified Foundations Associate – Java)	56
A.1	Course Details	56
A.2	Course Content	56
A.3	Hands-on Practice: Product Management Application	57
A.4	Learning Outcomes and Relevance to the Internship	57
A.5	Certificate	58

LEARNING OBJECTIVES / INTERNSHIP OBJECTIVES

The original objectives of the internship, as set out by the AI/ML Internship Program and the learning outcomes of its modules, were:

1. To build a strong foundation in Python programming for data science, including data types, control flow, functions and modular code.
2. To load, clean, analyse and visualise real-world data sets using NumPy, Pandas, Matplotlib and Seaborn, and to engineer features suitable for machine learning.
3. To understand the mathematics behind machine learning, namely linear algebra, statistics and probability, and use it to interpret data and models.
4. To build, evaluate and compare supervised learning models for regression and classification with scikit-learn, using appropriate metrics such as MAE, RMSE, R^2 , accuracy, precision, recall, F1-score and ROC-AUC.
5. To apply unsupervised learning techniques, K-Means clustering and Principal Component Analysis, for segmentation and dimensionality reduction.
6. To design and train artificial and convolutional neural networks with TensorFlow/Keras for image classification tasks.
7. To understand NLP fundamentals, prompt engineering and Retrieval-Augmented Generation, and to build document question-answering chatbots using large language models.
8. To build recommendation systems and to train reinforcement learning agents with Gymnasium and Stable-Baselines3.
9. To save trained models with Pickle or Joblib, develop REST APIs and web interfaces with Flask, containerise applications with Docker and deploy them on Render through GitHub.
10. To understand the fundamentals of MLOps (model packaging, version control, experiment tracking and continuous integration) and to follow professional practices of documentation and timely submission.

WEEKLY OVERVIEW OF INTERNSHIP ACTIVITIES

Week 1: Orientation and Python Fundamentals

1st WEEK	DATE	DAY	NAME OF THE TOPIC / MODULE COMPLETED
	20/05/2026	Wednesday	Orientation and induction to the AI/ML Internship Program (AI3, Batch 1A): structure, schedule, assignments and GitHub submission process; introduction to Artificial Intelligence, Machine Learning and Deep Learning and their applications
	21/05/2026	Thursday	Setting up the development environment (Anaconda, Jupyter Notebook, VS Code); Python basics: variables and data types — numbers, strings, lists, tuples, dictionaries and sets
	22/05/2026	Friday	Python operators: arithmetic, comparison, logical, assignment, membership and identity operators; operator precedence; practice programs

Summary of the week

The first week introduced the AI/ML Internship Program, its schedule of daily sessions from 1:00 pm to 3:00 pm, and the process for submitting assignments through public GitHub repositories and the program's submission form. The opening session gave an overview of artificial intelligence, machine learning and deep learning and their applications in industry and governance. The development environment was then set up with Anaconda, Jupyter Notebook and VS Code, and Python programming began with data types and operators, the building blocks of every program written later in the internship.

Week 2: Python Control Flow, Functions, NumPy and Pandas

2nd WEEK	DATE	DAY	NAME OF THE TOPIC / MODULE COMPLETED
	25/05/2026	Monday	Conditional statements: if, if-else, elif ladder, nested conditions and match-case; decision-making programs such as grading, tax slabs and a three-attempt login system
	26/05/2026	Tuesday	Loops: for and while loops, range(), break, continue and pass, nested loops; pattern and number programs (prime numbers, factorial, Fibonacci series)
	27/05/2026	Wednesday	Functions: definition, parameters and arguments, return values, default and keyword arguments, variable-length arguments, lambda functions, recursion and variable scope
	28/05/2026	Thursday	NumPy: creating arrays, indexing and slicing, reshaping, broadcasting, vectorised operations, statistical and linear-algebra functions
	29/05/2026	Friday	Pandas I: Series and DataFrame, reading CSV files, inspecting data (head, info, describe), selection with loc and iloc, filtering and sorting

Summary of the week

The second week completed the Python foundations. Conditional statements and loops were practised through a graded set of problems ranging from simple number programs to logic-based tasks such as an ATM menu and a three-attempt login system. Functions introduced reusable, modular code, including default and keyword arguments, lambda functions and recursion. The week ended with NumPy for fast numerical computing on arrays and the first part of Pandas for loading and exploring tabular data, which together form the basis of all data work in machine learning.

Week 3: Data Handling, Mathematics for ML, EDA and Feature Engineering

3rd WEEK	DATE	DAY	NAME OF THE TOPIC / MODULE COMPLETED
	01/06/2026	Monday	Pandas II: handling missing values and duplicates, groupby and aggregation, merging and joining DataFrames, apply and map
	02/06/2026	Tuesday	Mathematics for ML I: scalars, vectors and matrices, vector and matrix operations; representing a data set as a feature matrix X and a target vector y
	03/06/2026	Wednesday	Mathematics for ML II: descriptive statistics, probability and conditional probability; end-of-week mini project on the Titanic data set
	04/06/2026	Thursday	Exploratory Data Analysis: univariate and bivariate analysis, histograms, box plots, scatter plots and correlation heatmaps with Matplotlib and Seaborn; outlier detection
	05/06/2026	Friday	Feature engineering: label and one-hot encoding, feature scaling with StandardScaler, binning with <code>pd.cut()</code> , date-feature extraction and creation of new features

Summary of the week

The third week focused on preparing data for machine learning. Data cleaning in Pandas covered missing values, duplicates, grouping and merging. The mathematics sessions explained how a data set is represented as a feature matrix and a target vector, and reviewed the statistics and probability needed to interpret data and models, ending with a Titanic mini project. Exploratory data analysis with Matplotlib and Seaborn and feature engineering techniques such as encoding, scaling and binning completed the preparation needed before building models.

Week 4: Supervised Machine Learning using Linear Regression

4th WEEK	DATE	DAY	NAME OF THE TOPIC / MODULE COMPLETED
	08/06/2026	Monday	Introduction to supervised learning: the machine learning workflow, regression versus classification, training and testing data, train_test_split
	09/06/2026	Tuesday	Simple linear regression: equation of a line, cost function (mean squared error), gradient-descent intuition; implementation with scikit-learn LinearRegression
	10/06/2026	Wednesday	Multiple linear regression: several input features, interpreting coefficients and intercept, making predictions
	11/06/2026	Thursday	Evaluating regression models: MAE, MSE, RMSE and R^2 score; actual-versus-predicted and residual plots
	12/06/2026	Friday	Regression mini projects: house price prediction and student performance prediction; employee salary prediction system with feature engineering and scaling

Summary of the week

The fourth week introduced supervised machine learning through linear regression. After an overview of the machine learning workflow and the difference between regression and classification, simple and multiple linear regression were implemented with scikit-learn, and the meaning of the coefficients, the cost function and gradient descent was discussed. Regression models were evaluated with MAE, MSE, RMSE and R^2 together with actual-versus-predicted and residual plots, and the week closed with industry-style mini projects on house prices, student performance and employee salaries.

Week 5: Polynomial Regression, Classification and KNN

5th WEEK	DATE	DAY	NAME OF THE TOPIC / MODULE COMPLETED
	15/06/2026	Monday	Polynomial regression: modelling non-linear relationships, PolynomialFeatures, choosing the degree, underfitting and overfitting
	16/06/2026	Tuesday	Logistic regression: classification problems, the sigmoid function, log-odds, decision boundary and log loss; implementation with scikit-learn
	17/06/2026	Wednesday	Model evaluation for classification: confusion matrix, accuracy, precision, recall, F1-score, ROC curve and AUC, classification_report()
	18/06/2026	Thursday	K-Nearest Neighbors: distance metrics, choosing K, the importance of feature scaling, KNN for classification and regression
	19/06/2026	Friday	KNN practice: accuracy-versus-K plots and best-K search on the Iris and Heart Disease data sets

Summary of the week

The fifth week extended supervised learning. Polynomial regression showed how linear models can fit curves and how too high a degree causes overfitting. The focus then moved to classification: logistic regression and the sigmoid function, and the full set of classification metrics built on the confusion matrix, including precision, recall, F1-score and ROC-AUC. K-Nearest Neighbors introduced distance-based learning and the importance of feature scaling, and was practised by searching for the best value of K on real data sets.

Week 6: Tree-Based Models, SVM and Unsupervised Learning

6th WEEK	DATE	DAY	NAME OF THE TOPIC / MODULE COMPLETED
	22/06/2026	Monday	Decision trees: entropy, information gain and Gini impurity, the CART algorithm, tree depth and pruning
	23/06/2026	Tuesday	Random forest: bagging, random feature selection, voting and averaging, feature importance
	24/06/2026	Wednesday	Support Vector Machines: hyperplane, margin and support vectors, linear and RBF kernels, parameters C and gamma; comparison of five classifiers on the Adult Census Income data set
	25/06/2026	Thursday	Unsupervised learning: K-Means clustering, within-cluster sum of squares (WCSS) and the elbow method; hierarchical clustering and dendrograms
	26/06/2026	Friday	Principal Component Analysis: covariance, principal components, explained variance ratio and 2-D visualisation of high-dimensional data

Summary of the week

The sixth week covered tree-based models, support vector machines and unsupervised learning. Decision trees were studied through entropy, information gain and Gini impurity, and random forests showed how an ensemble of trees reduces variance and provides feature importance. Support vector machines with linear and RBF kernels were compared with other classifiers on the Adult Census Income data set. The week ended with unsupervised learning: K-Means and hierarchical clustering for grouping unlabelled data, and Principal Component Analysis for reducing dimensions and visualising data.

Week 7: Model Deployment, Docker and Introduction to Reinforcement Learning

7th WEEK	DATE	DAY	NAME OF THE TOPIC / MODULE COMPLETED
	29/06/2026	Monday	Model deployment concepts: the ML lifecycle, ML pipelines, saving and loading models with Pickle and Joblib
	30/06/2026	Tuesday	Flask: routes, request handling, HTML templates and forms, REST APIs returning JSON; running the Car Price Prediction web application locally
	01/07/2026	Wednesday	Deploying ML web applications: Git and GitHub, requirements.txt, Procfile and Unicorn; Car Price Prediction System deployed live on Render
	02/07/2026	Thursday	Docker: images, containers, Dockerfile, layer caching and port mapping; Dockerised Car Price and Student Placement Predictor applications
	03/07/2026	Friday	Introduction to Reinforcement Learning: agent, environment, state, action, reward and policy; Gymnasium and Stable-Baselines3; PPO agent for CartPole-v1

Summary of the week

The seventh week marked the transition from building models to delivering them. Models were saved with Pickle and Joblib and served through Flask routes, HTML forms and JSON APIs. The Car Price Prediction System, the main project of this report, was run locally, pushed to GitHub and deployed live on Render with Unicorn. Docker was then used to containerise the car price and student placement applications. The week ended with an introduction to reinforcement learning, in which a PPO agent was trained with Stable-Baselines3 to balance the CartPole.

Week 8: MLOps, Industry Practices and Capstone Projects

8th WEEK	DATE	DAY	NAME OF THE TOPIC / MODULE COMPLETED
	06/07/2026	Monday	MLOps: experiment tracking with MLflow, continuous integration and deployment with GitHub Actions, the pull-request and code-review workflow
	07/07/2026	Tuesday	End-to-end example: training an Iris classifier, serving it through a Flask API with a web interface, containerising it with Docker and deploying it on Render
	08/07/2026	Wednesday	Reinforcement learning project: analysis of the LunarLander-v3 environment (8-D observations, 4 discrete actions), PPO training for 500,000 timesteps, evaluation and video recording
	09/07/2026	Thursday	Recommendation systems: content-based and collaborative filtering, TF-IDF and cosine similarity, SVD on the MovieLens data; Movie Recommendation web application
	10/07/2026	Friday	Retrieval-Augmented Generation I: chatbots, document loading, chunking, embeddings, vector databases, retriever and LLM; document question-answering chatbot with Google Gemini

Summary of the week

The eighth week covered MLOps, industry practices and capstone-style projects. MLflow experiment tracking and GitHub Actions for continuous integration showed how teams test and ship ML code. An end-to-end Iris classifier was trained, served through a Flask API and web page, and containerised with Docker. A PPO agent was trained for 500,000 timesteps to land a spacecraft in LunarLander-v3. Recommendation systems were implemented as a movie recommender web application, and the first RAG session produced a document question-answering chatbot using Google Gemini.

Week 9: Deep Learning and Natural Language Processing

9th WEEK	DATE	DAY	NAME OF THE TOPIC / MODULE COMPLETED
	13/07/2026	Monday	Artificial Neural Networks: perceptron, layers, activation functions (ReLU, sigmoid, softmax), forward and backward propagation, loss functions and optimisers
	14/07/2026	Tuesday	Building ANNs with TensorFlow/Keras: Sequential model, compile and fit, epochs, batch size and validation split; MNIST handwritten digit classifier
	15/07/2026	Wednesday	Convolutional Neural Networks: convolution, kernels, stride and padding, feature maps, max and average pooling, CNN architecture
	16/07/2026	Thursday	CNN implementation: CIFAR-10 image classification with data augmentation, batch normalisation, dropout and learning-rate scheduling
	17/07/2026	Friday	Natural Language Processing: tokenisation, stop-word removal, stemming and lemmatisation, bag-of-words and TF-IDF, sentiment analysis and chatbot basics

Summary of the week

The ninth week was devoted to deep learning and natural language processing. Artificial neural networks were studied from the perceptron to backpropagation and optimisers, and implemented in TensorFlow/Keras on the MNIST handwritten digits. Convolutional neural networks introduced convolution, pooling and feature maps, and were applied to CIFAR-10 with data augmentation, batch normalisation and dropout. The week ended with NLP fundamentals, from text preprocessing and TF-IDF to sentiment analysis and the basics of chatbots.

Week 10: Prompt Engineering, Advanced RAG and Assignments 1–4

10th WEEK	DATE	DAY	NAME OF THE TOPIC / MODULE COMPLETED
	20/07/2026	Monday	Prompt engineering: zero-shot, few-shot and role prompting, step-by-step reasoning and structured outputs; Assignment 1 (Insurance Cost Prediction, Multiple Linear Regression) submitted
	21/07/2026	Tuesday	RAG II: answering from multiple documents, conversation memory and source citations; Assignment 2 (Customer Churn Prediction, Logistic Regression) submitted
	22/07/2026	Wednesday	RAG III: advanced techniques — history-aware retrieval and query rewriting, tuning chunk size, overlap and top-k, grounding prompts to reduce hallucination
	23/07/2026	Thursday	RAG IV: production-level project structure with separate indexing and querying modules; Assignment 3 (Salary Prediction, Polynomial Regression) and Assignment 4 (Breast Cancer Classification, KNN) submitted
	24/07/2026	Friday	Deep learning practice projects: Brain Tumour MRI classification (4 classes) and LFW face recognition (7 classes) with CNNs, early stopping and learning-rate reduction

Summary of the week

The tenth week combined advanced generative-AI topics with the graded assignment series. Prompt engineering techniques were followed by three further RAG sessions covering multiple documents, conversation memory, source citations, history-aware retrieval and a production-level project structure. Deep learning practice continued with CNN projects on brain tumour MRI scans and face recognition. In parallel, the first four graded assignments, on multiple linear regression, logistic regression, polynomial regression and KNN, were submitted.

Week 11: Assignments 5–10, Portfolio and Completion

11th WEEK	DATE	DAY	NAME OF THE TOPIC / MODULE COMPLETED
	27/07/2026	Monday	Assignment 5 (Employee Attrition Prediction using Decision Tree and Random Forest) and Assignment 6 (Weather Condition Classification using SVM with Open-Meteo API data) submitted
	28/07/2026	Tuesday	Assignment 7 (Customer Segmentation using K-Means Clustering and PCA) submitted
	29/07/2026	Wednesday	Amazon quarterly report (10-Q) RAG chatbot with FastAPI, sentence-transformers, FAISS and Gemini; consolidation of all projects into a portfolio repository
	30/07/2026	Thursday	Assignment 8 (Handwritten Digit Recognition using ANN) and Assignment 9 (Cats vs Dogs Image Classification using CNN) submitted
	31/07/2026	Friday	Assignment 10 (End-to-End Heart Disease Prediction: model training, Flask REST API, GitHub and Render deployment; final submission on 01/08/2026); completion of the internship

Summary of the week

The final week completed the remaining six graded assignments: a decision tree and random forest comparison, SVM classification on live weather data, customer segmentation with K-Means and PCA, digit recognition with an ANN, image classification with a CNN and, finally, an end-to-end heart disease prediction REST API deployed on Render. An Amazon quarterly-report RAG chatbot was also built with FastAPI and FAISS, and all projects were consolidated into a single portfolio repository. The internship concluded on 31st July, 2026.

1. INTRODUCTION

1.1 Artificial Intelligence and Machine Learning: An Overview

Artificial Intelligence (AI) is the branch of computer science that builds systems able to perform tasks which normally require human intelligence, such as recognising images, understanding language and making predictions. Machine Learning (ML) is the part of AI in which this behaviour is learned from data rather than programmed rule by rule: a model adjusts its parameters to reduce its errors on examples and then generalises to new cases. Deep Learning (DL) is a subset of ML that uses multi-layer neural networks and drives most recent progress in vision, speech and language [4].

Machine learning problems fall into three families [1], [3]. In **supervised learning**, every example carries the correct answer (the label), and the model learns to predict either a number (regression, such as the resale price of a car) or a category (classification, such as whether a customer will leave a service). In **unsupervised learning** there are no labels, and the aim is to discover structure, such as groups of similar customers (clustering) or a compact representation of many features (dimensionality reduction). In **reinforcement learning**, an agent learns by trial and error from the rewards it receives from an environment [6].

Large language models (LLMs) now make it possible to build assistants that answer questions in natural language. Because an LLM on its own can give confident but incorrect answers, Retrieval-Augmented Generation (RAG) first retrieves the passages of a document collection most relevant to a question and then asks the model to answer only from them [12], which makes chatbots over an organisation's own documents practical.

Finally, a model creates value only when it is deployed so that people and other systems can use it; MLOps (Machine Learning Operations) covers saving models, serving them through APIs, containerising applications and automating testing and deployment. The internship covered this complete path.

1.2 About the Organisation: MPOnline Limited

MPOnline Limited is the official e-governance service delivery organisation of the Government of Madhya Pradesh. It was incorporated in 2006 as a joint venture between

the Government of Madhya Pradesh, acting through the Madhya Pradesh State Electronics Development Corporation (MPSEDC), and Tata Consultancy Services (TCS), and is based in Bhopal [21]. It develops and operates the citizen services portal www.mponline.gov.in, guided by the idea of moving from "citizen in line" to "citizen online".

Through a single portal, MPOnline delivers Government-to-Citizen (G2C), Government-to-Business (G2B), Business-to-Citizen (B2C) and Government-to-Government (G2G) services, including online applications and payments, examination and recruitment processes, admission counselling, licences, certificates and state welfare schemes. A large statewide network of authorised kiosks and common service centres extends these services to citizens in rural areas.

The organisation is ISO certified and appraised at CMMI Maturity Level 3. Its large volumes of applications, examination records and payments are the kind of structured data to which machine learning applies, for example in forecasting service demand and answering citizen queries. MPOnline also conducts technology internships for students, including the AI/ML Internship Program described in this report.

1.3 About the AI/ML Internship Program

I completed the **AI/ML Internship Program (program code AI3)** at MPOnline Limited from **20th May, 2026 to 31st July, 2026** as a member of **Batch 1A** (application number IN26011052). Sessions were held from Monday to Friday between 1:00 pm and 3:00 pm and were conducted by the instructor, Nishant Shrivastava. The internship spanned 73 calendar days, comprising 53 working days, which is more than the minimum internship duration of 45 days required by the university. On completion, MPOnline issued a Certificate of Completion (Certificate ID: MPO/INT/26-27/07/359/26050363) signed by its Chief Technical Officer.

Each module combined a concept session supported by detailed notes and tutorials (53 sets over the program) [22], guided implementation in Python with Jupyter Notebook and VS Code, and practice work ranging from exercises to complete applications.

Assessment was continuous and practical: each of the ten graded assignments (10 marks each) was submitted before a fixed deadline as a public GitHub repository with a

README giving the student's registration, application and batch numbers, and the final assignment also required a live deployment on Render.

1.4 Tools and Technologies Used

Table 1.1 lists the tools used during the internship; all are open source or offer free tiers, so every project was built and deployed without special hardware or paid infrastructure.

Table 1.1: Tools and technologies used during the internship

Category	Tools, libraries and platforms
Language and environment	Python 3.10 – 3.12; Anaconda (Conda environments), Jupyter Notebook, VS Code
Data handling and visualisation	NumPy, Pandas, Matplotlib, Seaborn
Machine and deep learning	scikit-learn; TensorFlow and Keras
Reinforcement learning	Gymnasium (with Box2D), Stable-Baselines3, PyTorch
NLP and generative AI	LangChain, ChromaDB, FAISS, sentence-transformers, Google Gemini API, Streamlit
Web, deployment and MLOps	Flask, FastAPI, Gunicorn, Uvicorn, HTML/CSS, Tailwind CSS; Git, GitHub, Docker, Render; MLflow and GitHub Actions (covered in sessions)

1.5 Organisation of the Report

After the front matter, Chapter 1 introduces the field, the organisation and the program. Chapter 2 discusses how the objectives were achieved, the technical learning, the main project (the Car Price Prediction System), the assignments and other projects, and the results, skills and challenges. Chapter 3 concludes the report, Chapter 4 lists the references, and Annexure A summarises the Oracle Certified Foundations Associate (Java) course completed through PAT.

2. INTERNSHIP DISCUSSION

This chapter discusses the internship in detail. It first maps each original objective to the work that achieved it and then describes what was learned in each technical module. The main project, the Car Price Prediction System, is presented next, followed by the ten graded assignments and the additional projects with their results. The chapter closes with the observations drawn from the work, the scientific and professional skills gained, the challenges faced and how they were resolved, and the links to all project repositories.

2.1 How the Internship Objectives Were Achieved

The objectives listed at the beginning of this report were achieved through a combination of instructor-led sessions, hands-on implementation and graded work that was submitted publicly. Table 2.1 maps each objective to the activities that achieved it and to the evidence produced, and Figure 2.1 shows when each module was covered over the eleven weeks of the internship.

Table 2.1: Mapping of internship objectives to activities and evidence

Objective	How it was achieved	Evidence produced
1. Python foundations	Sessions on data types, operators, conditionals, loops and functions with graded practice programs (Weeks 1–2)	Python used in every notebook, script and application of the internship
2. Data handling and EDA	NumPy, Pandas, EDA and feature engineering sessions (Weeks 2–3)	Data cleaning and EDA steps in every assignment notebook
3. Mathematics for ML	Linear algebra, statistics and probability sessions with a Titanic mini project (Week 3)	Correct interpretation of metrics, explained variance and model outputs
4. Supervised learning	Regression and classification modules (Weeks 4–6)	Assignments 1–6; Car Price, Student Placement, Heart Disease and Adult Census projects
5. Unsupervised learning	K-Means, hierarchical clustering and PCA (Week 6)	Assignment 7: customer segmentation
6. Deep learning	ANN and CNN modules with TensorFlow/Keras (Weeks 9–10)	Assignments 8–9; CIFAR-10, Brain Tumour MRI and LFW projects

Objective	How it was achieved	Evidence produced
7. NLP, prompt engineering and RAG	NLP, prompt engineering and four RAG sessions (Weeks 8–10)	Document Knowledge Base chatbot; Amazon 10-Q RAG chatbot
8. Recommenders and RL	Recommendation-system and reinforcement-learning modules (Weeks 7–8)	Movie Recommendation System; CartPole and Lunar Lander PPO agents
9. Deployment	Pickle/Joblib, Flask, GitHub, Render and Docker sessions (Weeks 7–8)	Live Car Price, Student Placement, Movie Recommendation and Heart Disease services; Dockerised applications
10. MLOps and professional practice	MLOps session; version control and deadline-driven submissions (Weeks 8–11)	19 public GitHub repositories with README documentation; all ten graded assignments submitted

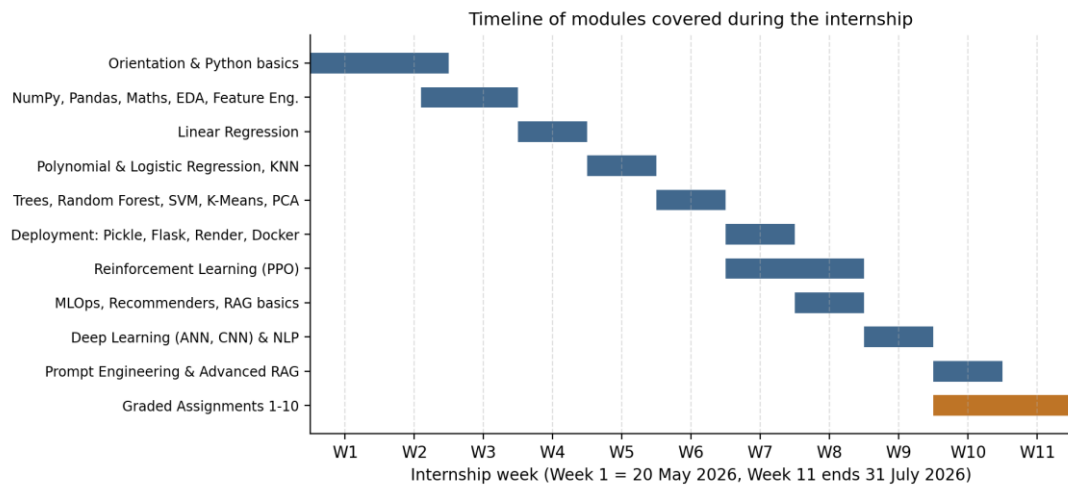


Figure 2.1: Timeline of the modules covered during the internship (reconstructed from the session notes and repository commit dates)

2.2 Technical Learning During the Internship

The program was organised as a sequence of modules, each building on the previous one. Almost every exercise and project followed the general machine learning workflow shown in Figure 2.2: a problem is defined, data is collected, cleaned and explored, features are engineered, a model is trained and evaluated (repeating these steps until its performance is acceptable), and the final model is saved and deployed. The subsections below summarise what was learned in each module.

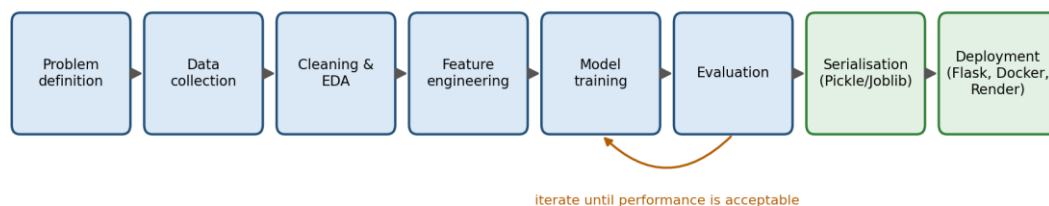


Figure 2.2: The machine learning workflow followed throughout the internship

2.2.1 Python Programming Foundations

The first two weeks built fluency in Python, the language used for the rest of the program. The sessions covered the built-in data types (numbers, strings, lists, tuples, dictionaries and sets) and their typical uses; arithmetic, comparison, logical, membership and identity operators; decision making with if, elif and match-case; iteration with for and while loops together with break and continue; and functions with default and keyword arguments, variable-length arguments, lambda expressions and recursion. Each topic ended with graded practice questions, ranging from simple programs such as even numbers, factorials and multiplication tables to logic-based problems such as an ATM menu, a login system that allows only three attempts, income-tax slabs and pattern printing. These exercises built the habit of breaking a problem into small, testable steps, which later made data-processing code easier to write and to debug.

2.2.2 Numerical Computing, Data Analysis and Mathematics for ML

NumPy introduced the n-dimensional array, vectorised arithmetic and broadcasting, which replace slow Python loops with fast array operations and form the foundation of every ML library used later. Pandas introduced the Series and the DataFrame: reading CSV files, selecting rows and columns with loc and iloc, filtering, sorting, handling missing values and duplicates, grouping and aggregation, and merging data sets [2]. The mathematics module connected these tools to machine learning theory. A data set is a matrix X of features together with a target vector y , and a model is a function that maps the rows of X to predictions. Statistics such as the mean, median and standard deviation, together with probability and conditional probability, describe the data before any model is built. The module ended with a Titanic mini project in which

survival probabilities, conditional probabilities by gender and simple baseline predictions evaluated with MAE and MSE were computed.

2.2.3 Exploratory Data Analysis and Feature Engineering

Exploratory Data Analysis (EDA) is the step in which the data is understood before any modelling. The sessions covered inspecting the shape, data types and summary statistics of a data set; univariate analysis with histograms and box plots; bivariate analysis with scatter plots; correlation heatmaps; and the detection of outliers. Feature engineering then prepared the data for algorithms: encoding categorical variables with label encoding or one-hot encoding (using `pd.get_dummies` with `drop_first` to avoid redundant columns), scaling numerical features with `StandardScaler`, creating new features from existing ones (for example, a car's age from its year of manufacture), binning continuous values into categories with `pd.cut` and extracting parts of dates. A key lesson was that most of the effort in a real project goes into these steps, and that the choices made here often matter more than the choice of algorithm.

2.2.4 Supervised Learning: Regression

Regression predicts a continuous value. Simple linear regression fits a straight line, $y = mx + c$, by minimising the mean squared error between predictions and actual values, and multiple linear regression extends this to many input features, where each coefficient represents the change in the output for a unit change in one feature while the others are held constant [3]. Polynomial regression adds powers of a feature (x^2 , x^3 and so on) so that a linear model can fit curves; the degree controls the flexibility of the model, and too high a degree leads to overfitting. Models were evaluated with the MAE (the average absolute error), the MSE and RMSE (which penalise large errors more heavily, the RMSE being in the same units as the target) and the R^2 score (the proportion of the variance in the target that the model explains). Mini projects on house prices, student performance and employee salaries, with engineered features, scaling and residual plots, gave practice in reading these metrics together rather than relying on any single number.

2.2.5 Supervised Learning: Classification and Model Evaluation

Classification predicts a category. Logistic regression passes a linear combination of the features through the sigmoid function to produce a probability between 0 and 1, and

a threshold (0.5 by default) converts this probability into a class. K-Nearest Neighbors (KNN) classifies a new point by a majority vote among the K most similar training points, which makes feature scaling essential, because otherwise the distances are dominated by the features with the largest ranges. The evaluation module explained why accuracy alone can be misleading, especially when one class is rare, and introduced the confusion matrix and the metrics derived from it: **precision** (of the cases predicted positive, how many were correct), **recall** (of the actual positive cases, how many were found), the **F1-score** (the harmonic mean of precision and recall) and the **ROC curve** with its area under the curve (AUC), which summarises performance across all possible thresholds.

2.2.6 Tree-Based Models, Ensembles and Support Vector Machines

A decision tree repeatedly splits the data on the feature and threshold that best separate the classes, measured with entropy (information gain) or Gini impurity. Trees are easy to interpret but have high variance: small changes in the data can produce very different trees, and an unrestricted tree memorises its training data. Random forests reduce this variance by training many trees on bootstrap samples of the data, with a random subset of features considered at each split, and by combining their votes for classification or averaging their outputs for regression; they also provide feature-importance scores [8]. Support Vector Machines find the separating hyperplane with the widest margin between the classes, and kernels such as the RBF kernel allow non-linear boundaries, with the parameters C and gamma controlling the trade-off between a smooth boundary and a close fit to the training data [9]. Five classifiers were compared side by side on the Adult Census Income data set to see these trade-offs in practice (Section 2.5.7).

2.2.7 Unsupervised Learning and Dimensionality Reduction

K-Means clustering partitions data into K groups by repeatedly assigning each point to the nearest centroid and recomputing the centroids, which minimises the within-cluster sum of squares (WCSS). The elbow method plots the WCSS against K to help choose the number of clusters. Hierarchical (agglomerative) clustering instead merges points from the bottom up and records the merges in a dendrogram, so that K does not have to be fixed in advance. Principal Component Analysis (PCA) finds new, uncorrelated axes, called principal components, that capture the maximum variance in the data. Keeping only the first two or three components allows high-dimensional data to be

visualised and compressed, and the explained-variance ratio shows how much of the original information is retained.

2.2.8 Deep Learning: Artificial and Convolutional Neural Networks

An artificial neural network is built from layers of neurons, each computing a weighted sum of its inputs followed by a non-linear activation function such as ReLU, with a softmax output layer for multi-class problems. Training uses backpropagation to compute the gradients of the loss and an optimiser such as Adam [20] to update the weights over many epochs. Convolutional Neural Networks (CNNs) are designed for images: small learnable filters slide over the image to detect edges, textures and shapes, sharing their weights across positions, while pooling layers reduce the spatial size [16]. Using TensorFlow and Keras [5], [24], the sessions covered building Sequential models, choosing loss functions, monitoring training and validation curves, and controlling overfitting with data augmentation, dropout [18], batch normalisation [19], early stopping and learning-rate reduction.

2.2.9 NLP, Prompt Engineering and Retrieval-Augmented Generation

The NLP module covered text preprocessing (tokenisation, lower-casing, stop-word removal, stemming and lemmatisation), the representation of text as numbers with bag-of-words and TF-IDF, sentiment analysis and the basics of chatbots. Prompt engineering covered how to instruct large language models reliably: giving a clear role and context, zero-shot and few-shot prompting, step-by-step reasoning and requesting structured outputs. The RAG series, delivered in eight parts, built a document question-answering chatbot step by step: loading documents, splitting them into overlapping chunks, converting the chunks into embedding vectors (for example, the 384-dimensional vectors produced by the all-MiniLM-L6-v2 sentence-transformer model [14]), storing the vectors in a vector database such as FAISS [13], retrieving the chunks most similar to a question and passing them to the LLM with a prompt that restricts the answer to the retrieved context. Later parts added multiple documents, conversation memory, source citations, history-aware query rewriting and a modular, production-style project structure.

2.2.10 Recommendation Systems

Recommendation systems help users find relevant items among thousands of choices. Content-based filtering recommends items similar to those a user already likes, using item features such as genres, represented with TF-IDF and compared with cosine similarity. Collaborative filtering instead uses the ratings of many users, recommending items liked by similar users (user-based) or items rated similarly to the user's own choices (item-based). Matrix factorisation with Singular Value Decomposition (SVD) on the MovieLens rating matrix [15] learns latent factors for users and items and predicts the ratings a user would give to movies they have not yet seen. The module also discussed practical issues such as the cold-start problem for new users and items and the sparsity of rating data.

2.2.11 Reinforcement Learning

Reinforcement learning was taught through Gymnasium environments and the Stable-Baselines3 library [11], [28]. An agent observes the state of an environment, chooses an action, receives a reward and a new state, and learns a policy that maximises the cumulative discounted reward, where the discount factor γ controls how much future rewards matter. Proximal Policy Optimization (PPO) [10], an actor-critic algorithm that limits how far each policy update can move by clipping its objective, was used for both CartPole-v1 and LunarLander-v3. The module also covered choosing algorithms for different action spaces (PPO and DQN for discrete actions, SAC for continuous control), saving and reloading trained agents, evaluating them over many episodes and plotting learning curves.

2.2.12 Model Deployment, Docker and MLOps

The deployment module showed how a trained model becomes a usable service. Models were serialised with Pickle or Joblib, loaded once when a Flask application starts, and exposed through routes that either render an HTML form for people or accept and return JSON for other programs. In production, the Flask application runs behind Gunicorn, a WSGI server [25], and the exact versions of its dependencies are pinned in requirements.txt. Projects were pushed to GitHub and deployed on Render, which builds the application, starts it with the configured command, serves it at a public HTTPS address and can redeploy it automatically on every push. Docker packages an application together with its exact Python version and libraries into an image that runs

identically on any machine [26]. The MLOps session introduced experiment tracking with MLflow and continuous integration with GitHub Actions, in which tests written by the developer run automatically on every push or pull request before code is merged and deployed.

2.3 Main Project: Car Price Prediction System

The Car Price Prediction System was selected as the main project of this report because it covers the complete machine learning lifecycle taught during the internship in a compact and easily explained form: data acquisition, cleaning and feature engineering, model training and evaluation, serialisation, a web interface, cloud deployment and containerisation. The application is live at **<https://car-price-prediction-app-00au.onrender.com>**, and its source code is available in the GitHub repository **AkshatGarg2005/car-price-prediction-app**. The model was developed in a Jupyter notebook and first run locally as a Flask application on 30th June, 2026; it was deployed on Render on 1st July, 2026, and a Docker version was completed on 2nd July, 2026.

2.3.1 Problem Statement

The used-vehicle market in India is large, but prices are often decided by guesswork. A seller does not know what price to ask, and a buyer cannot easily judge whether an asking price is fair. The price of a used vehicle depends on several measurable factors: its original ex-showroom price, its age, the distance it has been driven, its fuel type and transmission, the number of previous owners and whether it is being sold by a dealer or by an individual. The problem addressed by this project is to learn this relationship from historical sales data and to provide an instant, data-driven estimate of the resale price through a simple web page that anyone can use.

2.3.2 Objectives of the Project

- To analyse the CarDekho used-vehicle data set and prepare it for machine learning.
- To engineer meaningful features, such as the age of the vehicle, and to encode categorical attributes as numbers.
- To train a regression model that predicts the selling price in lakh rupees, and to evaluate it on unseen data using the R^2 score and RMSE.

- To save the trained model and serve it through a Flask web application with an input form.
- To deploy the application publicly on Render and to package it as a Docker container so that it runs identically in any environment.

2.3.3 Software and Hardware Requirements

Table 2.2: Software requirements of the Car Price Prediction System

Component	Specification
Operating system (development)	macOS on an Apple Silicon Mac
Programming language	Python 3.10 (Conda environment named carprice)
Libraries	pandas 2.3.3, NumPy 2.2.6, scikit-learn 1.7.2, Flask 3.1.3, Gunicorn 23.0.0, Matplotlib, Seaborn, opendatasets 0.1.22
Development tools	Jupyter Notebook, VS Code, Git, Docker
Hosting	GitHub (source code) and Render web service (Linux)
Client	Any modern web browser on a computer or mobile phone

No special hardware is needed. Training the model on 301 records takes well under a second, and the saved model is about 2 MB. Development was carried out on an Apple Silicon Mac, but any computer able to run Python 3.10 is sufficient. The deployed service runs on a Render web instance, and an end user needs only a device with a web browser and an internet connection.

2.3.4 Dataset Description

The project uses the **Vehicle Dataset from CarDekho**, published on Kaggle [30], specifically the file car data.csv. It contains 301 records and 9 columns with no missing values. Table 2.3 describes the columns together with their ranges and category counts, computed during the analysis.

Table 2.3: Columns of the CarDekho data set (car data.csv)

Column	Type	Description and values
Car_Name	Text	Name of the model (98 distinct values)
Year	Integer	Year of manufacture, from 2003 to 2018
Selling_Price	Decimal	Target variable: price at which the vehicle was sold, in lakh rupees (0.10 to 35.00; mean 4.66)

Column	Type	Description and values
Present_Price	Decimal	Current ex-showroom price of the same model, in lakh rupees (0.32 to 92.60)
Kms_Driven	Integer	Distance driven in kilometres (500 to 500,000)
Fuel_Type	Category	Petrol (239), Diesel (60), CNG (2)
Seller_Type	Category	Dealer (195), Individual (106)
Transmission	Category	Manual (261), Automatic (40)
Owner	Integer	Number of previous owners: 0 (290 vehicles), 1 (10), 3 (1)

The exploratory analysis in Figure 2.3 shows that the selling price is strongly right-skewed: most vehicles sell for under ₹10 lakh and only a few premium models sell for up to ₹35 lakh. The selling price is strongly correlated with the present price (correlation coefficient 0.88), while age has a weaker negative relationship with it (−0.24). Three further observations shaped the interpretation of the model. First, although the file is named car data, 101 of the 301 records are two-wheelers, such as the Royal Enfield Classic 350 and the Honda Activa; all of them are sold by individual sellers for at most ₹1.75 lakh, so the Seller_Type column largely separates two-wheelers from cars. Second, the categories are imbalanced; for example, only two CNG vehicles are present, so predictions for CNG vehicles rest on very little data. Third, the file contains two exact duplicate rows; these were retained in the notebook and have a negligible effect on the results.

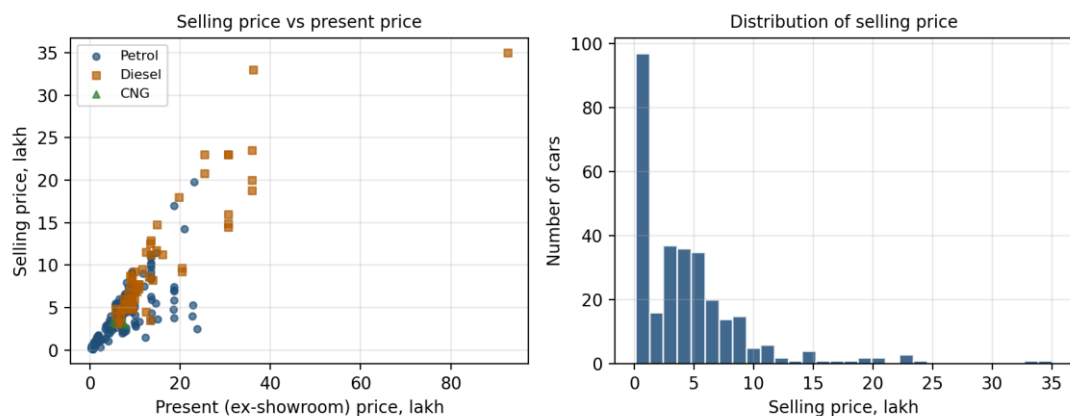


Figure 2.3: Selling price against present price by fuel type (left) and the distribution of selling prices (right)

2.3.5 Data Preprocessing and Feature Engineering

The following steps prepared the data for training (Listing 2.1):

1. The data was downloaded directly from Kaggle with the `opendatasets` library and loaded into a Pandas DataFrame, and its shape, data types, summary statistics and null counts were inspected.
2. A new feature, **Car_Age**, was created as $2025 - \text{Year}$, because the age of a vehicle is more directly related to depreciation than the calendar year. As the newest vehicles in the data are from 2018, the ages range from 7 to 22 years.
3. **Car_Name** was dropped because it has 98 distinct values for only 301 rows, which would create too many sparse columns if it were one-hot encoded, and **Year** was dropped because **Car_Age** replaces it.
4. The categorical columns were encoded with explicit mappings, so that the web form can send exactly the same codes: **Fuel_Type** (Petrol = 0, Diesel = 1, CNG = 2), **Seller_Type** (Dealer = 0, Individual = 1) and **Transmission** (Manual = 0, Automatic = 1).
5. The data was divided into the feature matrix **X** (**Present_Price**, **Kms_Driven**, **Fuel_Type**, **Seller_Type**, **Transmission**, **Owner** and **Car_Age**) and the target **y** (**Selling_Price**), and then split into 80% training data (240 vehicles) and 20% test data (61 vehicles) with `random_state = 42` so that the split is reproducible.

Feature scaling was not required, because tree-based models split on thresholds and are not affected by the scale of the features.

2.3.6 Model Development: Random Forest Regressor

A **Random Forest Regressor** from `scikit-learn` [7], [23] was chosen, with 100 trees (`n_estimators = 100`) and `random_state = 42`. A random forest is an ensemble of decision trees [8]. Each tree is trained on a bootstrap sample of the training rows, drawn with replacement, and at each split it considers a random subset of the features; for regression, the prediction of the forest is the average of the predictions of all its trees. Averaging many different trees reduces the variance of a single tree while keeping its ability to model non-linear relationships and interactions between features, for example that the effect of mileage on price depends on the original price of the vehicle.

A random forest suits this problem for several reasons: the relationship between price and the features is not linear; the data mixes continuous and categorical features; the model needs no feature scaling; it is robust to the outliers present in **Kms_Driven** and

Present_Price; and it provides feature-importance scores that make the model easier to explain. After training, the model was serialised with Python's pickle module to the file `car_price_model.pkl`, which was then reloaded and used for a test prediction to confirm that the saved file works before it was used in the web application.

2.3.7 Model Evaluation

The model was evaluated on the 61 test vehicles, which were not used in training. Table 2.4 lists the results. The R^2 score and RMSE were printed in the project notebook; the MAE, the training R^2 and the additional checks were computed afterwards from the same saved model, data and split (Listing 2.2).

Table 2.4: Evaluation of the Car Price Prediction model

Measure	Value	Interpretation
R^2 score (test set)	0.958	The model explains 95.8% of the variation in the selling prices of unseen vehicles
RMSE (test set)	0.984 lakh	Typical size of an error, with large errors weighted more (about ₹98,400)
MAE (test set)	0.654 lakh	Average absolute error (about ₹65,400)
R^2 score (training set)	0.985	The small gap from the test score shows limited overfitting
Linear Regression baseline (test R^2 / RMSE)	0.847 / 1.88 lakh	A straight-line model on the same features is clearly weaker
5-fold cross-validation R^2 (Random Forest)	0.886 – 0.959 (mean 0.920)	Performance across five different splits of all 301 records

Two additional checks were carried out to confirm that the result is reliable and that the random forest was a good choice. First, a plain Linear Regression model trained on the same features scored an R^2 of only 0.847 on the same test set, confirming that the random forest captures non-linear effects that a straight-line model cannot. Second, because a test set of 61 vehicles is small, 5-fold cross-validation was performed: the random forest achieved R^2 scores between 0.886 and 0.959 across the five folds, with a mean of 0.920. The test-set score of 0.958 is therefore at the favourable end of the model's range, and 0.92 is a more conservative estimate of its expected performance. Figure 2.4 compares predicted and actual prices on the test set; most points lie close to the diagonal, and the largest errors, of about ₹2.6 lakh, occur for the few expensive vehicles for which the training data is thin.

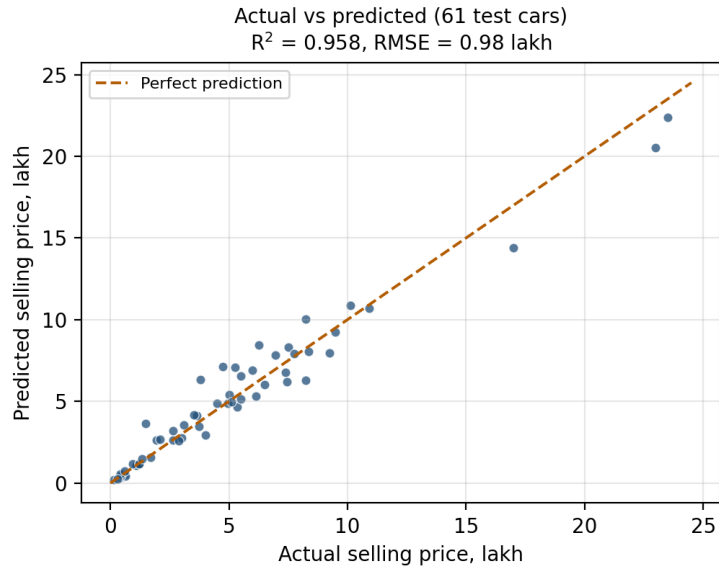


Figure 2.4: Actual versus predicted selling prices for the 61 test vehicles

Figure 2.5 shows the feature importances of the deployed model. `Present_Price` dominates with an importance of 0.885, followed by `Car_Age` (0.062) and `Kms_Driven` (0.033), while the remaining features contribute very little. This agrees with intuition, since the resale value of a vehicle is anchored to its original price and then reduced by age and usage, but it also shows that the model relies heavily on a single input.

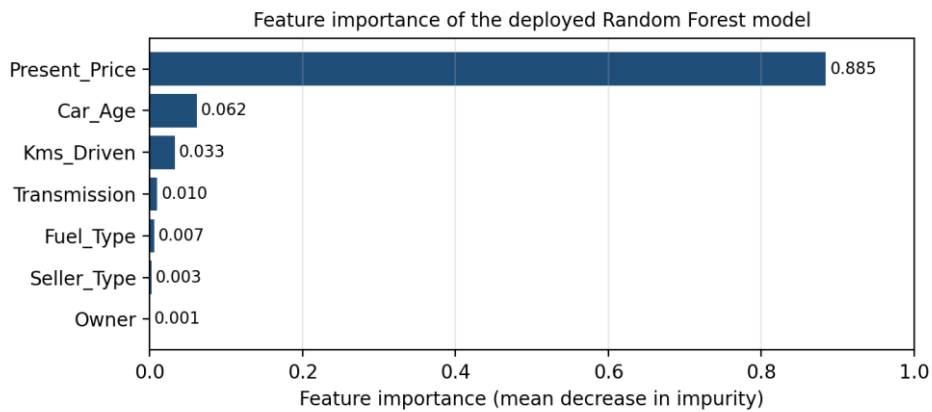


Figure 2.5: Feature importances of the deployed Random Forest model

2.3.8 System Architecture

Figure 2.6 shows the architecture of the deployed system. The user's browser sends an HTTP GET request to the root URL and receives the input form. When the form is submitted, the browser sends an HTTP POST request with the seven input values to the `/predict` route. On Render, the request is received by Gunicorn, a production-grade WSGI server, which passes it to the Flask application. Flask reads the form fields,

converts them to numbers in the same order that was used in training and calls the Random Forest model, which was loaded from `car_price_model.pkl` once when the application started, so no time is spent reloading it for each request. The predicted price is rounded to two decimal places and inserted into the same HTML template, which is returned to the browser.

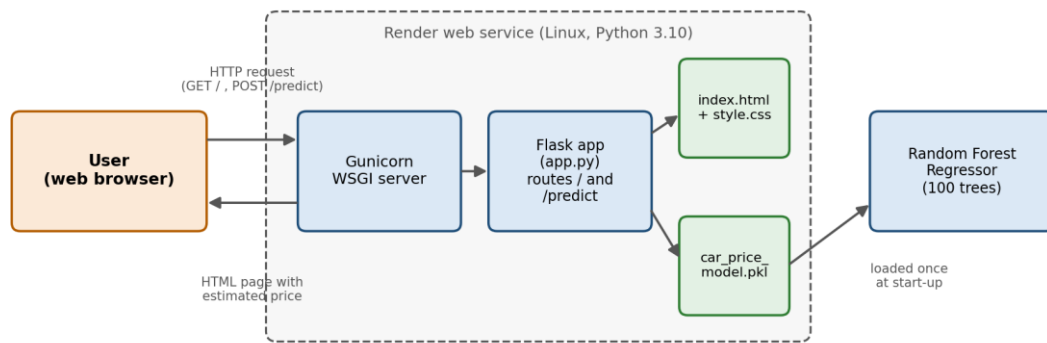


Figure 2.6: Architecture of the deployed Car Price Prediction System

2.3.9 Web Application using Flask

The web application (Listing 2.3) has two routes, listed in Table 2.5, and a single-page interface (Listing 2.4) whose input fields are listed in Table 2.6.

Table 2.5: Routes of the Flask application

Route	Method	Purpose
/	GET	Renders templates/index.html with the empty input form
/predict	POST	Reads the seven form fields, predicts the price and re-renders the page with "Estimated Car Price: ₹ X Lakhs"

Table 2.6: Input fields of the web form and the values sent to the model

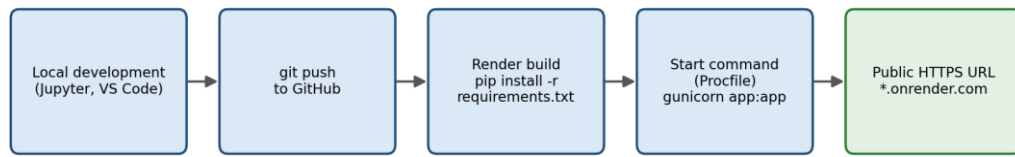
Field	Input type	Value sent to the model
Present Price (in Lakhs)	Number (decimal)	Value entered, for example 6.5
Kilometers Driven	Number	Value entered, for example 45000
Fuel Type	Drop-down list	Petrol = 0, Diesel = 1, CNG = 2
Seller Type	Drop-down list	Dealer = 0, Individual = 1
Transmission	Drop-down list	Manual = 0, Automatic = 1
Number of Previous Owners	Number	Value entered, for example 0
Car Age (in years)	Number	Value entered, for example 5

The interface is styled with a separate stylesheet (`static/style.css`) in a forest-green theme. All numeric fields are marked as required, so the browser does not submit an incomplete form. The prediction code is wrapped in a `try-except` block: if a value cannot be converted to a number, the page shows the message "Error: Please verify your input parameters." instead of failing. Because the drop-down options send numeric codes, the web form always uses exactly the same encoding as the training data.

2.3.10 Deployment on Render

The application was deployed on Render, a cloud platform that builds and hosts web services directly from a GitHub repository [27]. The deployment followed the pipeline shown in Figure 2.7:

1. The project folder, containing `app.py`, `car_price_model.pkl`, `templates/index.html`, `static/style.css`, `requirements.txt`, `runtime.txt` and `Procfile` (Listing 2.5), was committed with Git and pushed to the public GitHub repository `car-price-prediction-app` on 1st July, 2026.
2. `requirements.txt` pins the exact library versions used during development (Flask 3.1.3, NumPy 2.2.6, scikit-learn 1.7.2, pandas 2.3.3, Gunicorn 23.0.0 and opendatasets 0.1.22), so that the pickled model is loaded by the same version of scikit-learn that created it.
3. The Python version was kept at 3.10 to match the local Conda environment (`runtime.txt` records Python 3.10.16); Section 2.8 explains why matching the Python version on Render was necessary.
4. The `Procfile` contains the start command `web: gunicorn app:app`, which serves the Flask object named `app`, defined in `app.py`, through Gunicorn.
5. On Render, a new Web Service was created and connected to the repository. Render installed the requirements, started the service and assigned the public address `https://car-price-prediction-app-00au.onrender.com`. Later changes can be deployed automatically on each push or started manually from the dashboard; the README update, for example, was released with a manual deploy (Figure 2.9).



Each new commit can be redeployed automatically (auto-deploy) or manually from the Render dashboard

Figure 2.7: Deployment pipeline from local development to the live Render service

On Render's free plan, a service that receives no traffic for some time is put to sleep, so the first request after a pause can take up to about a minute while the service starts again; later requests are answered immediately.

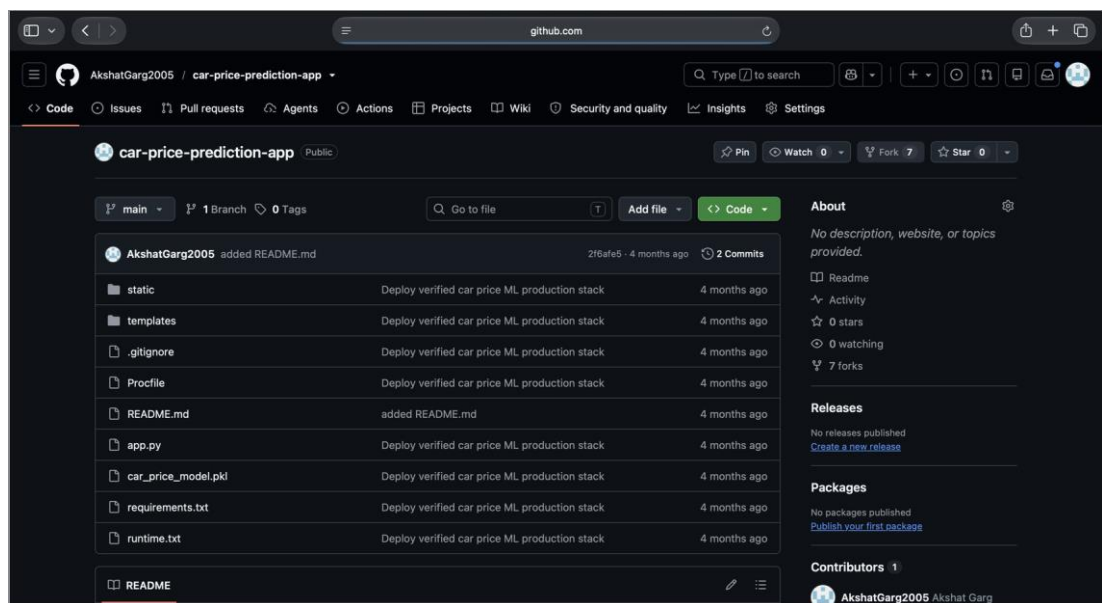


Figure 2.8: GitHub repository of the Car Price Prediction System

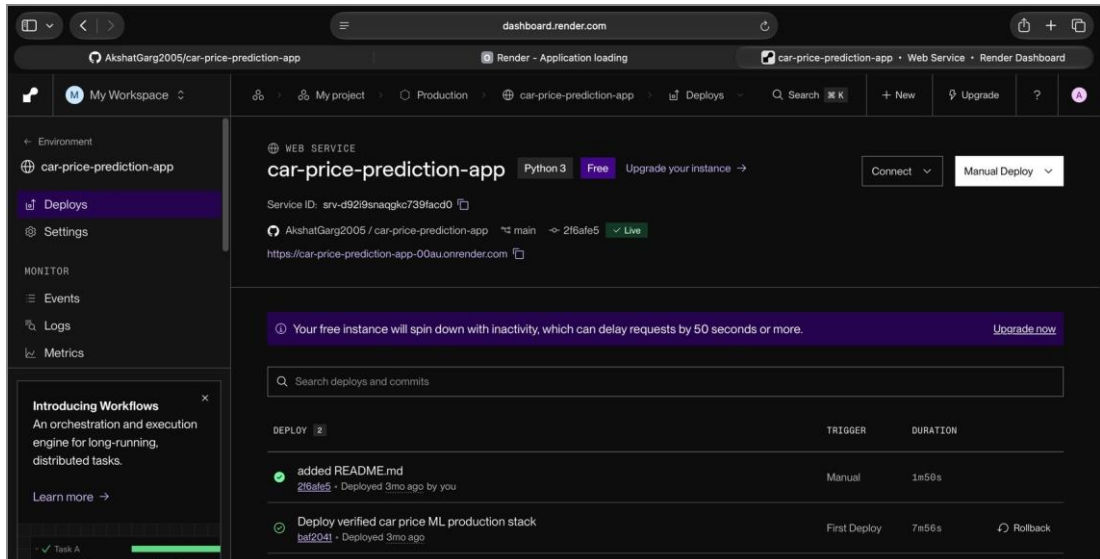


Figure 2.9: Render dashboard of the live web service on the free plan, with its public URL and deployment history

2.3.11 Containerised Version using Docker

To make the application independent of the hosting platform, a second version was packaged with Docker in the repository **car-price-predictor-app-docker** on 2nd July, 2026. It contains the same application code and the identical trained model, with a Dockerfile (Listing 2.6) in place of the Procfile and runtime.txt. The Dockerfile:

- starts from the official, lightweight python:3.10-slim image, which fixes the Python version inside the container;
- sets /app as the working directory;
- copies requirements.txt first and installs the dependencies with `pip install --no-cache-dir`, so that Docker can reuse this cached layer when only the application code changes and the image stays small;
- copies the rest of the project, exposes port 10000 (the default port used by Render) and starts Gunicorn bound to 0.0.0.0:10000, so that the server accepts connections from outside the container.

The image is built and started locally with the two commands at the end of Listing 2.6, after which the application is available at `http://localhost:10000`. The same image can be deployed on any container platform, including Render's Docker runtime, which was used for the Dockerised Student Placement Predictor (Section 2.5.4).

2.3.12 Source Code

The listings below give the complete source of the deployed application and its configuration files, together with the model-training code from the project notebook (lightly condensed) and the code used for the additional evaluation checks.

Listing 2.1: Model training code (from the project notebook)

```
import pandas as pd
import numpy as np
import pickle
import opendatasets as od
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score

# download the CarDekho data set from Kaggle and load it
url = "https://www.kaggle.com/datasets/nehalbirla/vehicle-dataset-from-cardekho"
od.download(url)
df = pd.read_csv("vehicle-dataset-from-cardekho/car data.csv")
print("shape:", df.shape)          # (301, 9)
print(df.isnull().sum())           # no null values

# feature engineering: age of the car instead of the year
df['Car_Age'] = 2025 - df['Year']
df.drop(['Car_Name', 'Year'], axis=1, inplace=True)

# encode the categorical columns manually (same codes as the web form)
df['Fuel_Type'] = df['Fuel_Type'].map({'Petrol': 0, 'Diesel': 1, 'CNG': 2})
df['Seller_Type'] = df['Seller_Type'].map({'Dealer': 0, 'Individual': 1})
df['Transmission'] = df['Transmission'].map({'Manual': 0, 'Automatic': 1})

# split into X and y, then 80-20 train-test split
X = df.drop('Selling_Price', axis=1)
y = df['Selling_Price']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# train the Random Forest and evaluate it on the test set
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("R2 score:", r2_score(y_test, y_pred))          # 0.9579
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))  # 0.9844

# save the model for the Flask application and check that it loads
pickle.dump(model, open('car price model.pkl', 'wb'))
loaded_model = pickle.load(open('car price model.pkl', 'rb'))
print("quick test prediction:", loaded_model.predict([X_test.iloc[0]]))
```

Listing 2.2: Additional evaluation checks (run after the training code)

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score, KFold
from sklearn.metrics import mean_absolute_error

# test-set MAE and training R2 of the same model
print("Test MAE:", mean_absolute_error(y_test, y_pred))          # 0.6535
print("Train R2:", r2_score(y_train, model.predict(X_train)))    # 0.9851

# baseline: plain Linear Regression on the same split
lr = LinearRegression().fit(X_train, y_train)
print("Linear Regression R2:", r2_score(y_test, lr.predict(X_test))) # 0.8468

# 5-fold cross-validation of the Random Forest on all 301 records
kf = KFold(n_splits=5, shuffle=True, random_state=42)
rf = RandomForestRegressor(n_estimators=100, random_state=42)
cv = cross_val_score(rf, X, y, cv=kf, scoring="r2")
print("5-fold R2:", cv.round(3), "mean:", cv.mean().round(3))    # mean 0.920

# feature importance of the trained model
print(dict(zip(X.columns, model.feature_importances_.round(3))))
```

Listing 2.3: app.py — Flask application (deployed version)

```
# app.py
from flask import Flask, render_template, request
import pickle
import numpy as np

app = Flask(__name__)

# Load the saved Random Forest model
with open('car_price_model.pkl', 'rb') as file:
    model = pickle.load(file)

@app.route('/')
def home():
    return render_template('index.html')

@app.route('/predict', methods=['POST'])
def predict():
    if request.method == 'POST':
        try:
            # Extract form input fields using their exact HTML name attributes
            present_price = float(request.form['present_price'])
            kms_driven = int(request.form['kms_driven'])
            fuel_type = int(request.form['fuel_type'])
            seller_type = int(request.form['seller_type'])
            transmission = int(request.form['transmission'])
            owner = int(request.form['owner'])
            car_age = int(request.form['car_age'])

            # Restructure inputs into a 2D array for the scikit-learn model
            features = np.array([[present_price, kms_driven, fuel_type, seller_type,
            transmission, owner, car_age]])

            # Predict price using the model pipeline
            prediction = model.predict(features)[0]
            output = round(prediction, 2)

            return render_template('index.html', prediction_text=f'Estimated Car
            Price: ₹ {output} Lakhs')
        except Exception as e:
            return render_template('index.html', prediction_text="Error: Please
            verify your input parameters.")
```

```
if __name__ == '__main__':
    app.run(debug=True)
```

Listing 2.4: templates/index.html — user interface

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Car Price Predictor</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
    <div class="container">
        <h1>Car Price Predictor</h1>
        <form action="/predict" method="POST">
            <label for="present_price">Present Price (in Lakhs)</label>
            <input type="number" step="any" id="present_price" name="present_price"
placeholder="e.g. 6.5" required>

            <label for="kms driven">Kilometers Driven</label>
            <input type="number" id="kms_driven" name="kms_driven" placeholder="e.g.
45000" required>

            <label for="fuel type">Fuel Type</label>
            <select id="fuel_type" name="fuel_type">
                <option value="0">Petrol</option>
                <option value="1">Diesel</option>
                <option value="2">CNG</option>
            </select>

            <label for="seller type">Seller Type</label>
            <select id="seller_type" name="seller_type">
                <option value="0">Dealer</option>
                <option value="1">Individual</option>
            </select>

            <label for="transmission">Transmission</label>
            <select id="transmission" name="transmission">
                <option value="0">Manual</option>
                <option value="1">Automatic</option>
            </select>

            <label for="owner">Number of Previous Owners</label>
            <input type="number" id="owner" name="owner" placeholder="e.g. 0"
required>

            <label for="car_age">Car Age (in years)</label>
            <input type="number" id="car_age" name="car_age" placeholder="e.g. 5"
required>

            <button type="submit" class="btn">Predict Price</button>
        </form>

        {% if prediction_text %}
        <div class="result">{{ prediction_text }}</div>
        {% endif %}
    </div>
</body>
</html>
```

The heading of the page in the repository also contains a car emoji before the title, which has been omitted from Listing 2.4 for printing.

Listing 2.5: Deployment configuration files: Procfile, runtime.txt and requirements.txt

```
# Procfile
web: gunicorn app:app

# runtime.txt
python-3.10.16

# requirements.txt
Flask==3.1.3
numpy==2.2.6
scikit-learn==1.7.2
pandas==2.3.3
gunicorn==23.0.0
opendatasets==0.1.22
```

Listing 2.6: Dockerfile of car-price-predictor-app-docker and the commands to build and run it

```
# Use an official, lightweight Python base image
FROM python:3.10-slim

# Set the working directory inside the container sandbox
WORKDIR /app

# Copy the requirements file first to optimize layer caching
COPY requirements.txt .

# Install dependencies inside the sandbox environment
RUN pip install --no-cache-dir -r requirements.txt

# Copy the remaining project files
COPY . .

# Expose port 10000 (Render default port)
EXPOSE 10000

# Run Gunicorn production server bound to Render's default port
CMD ["gunicorn", "--bind", "0.0.0.0:10000", "app:app"]

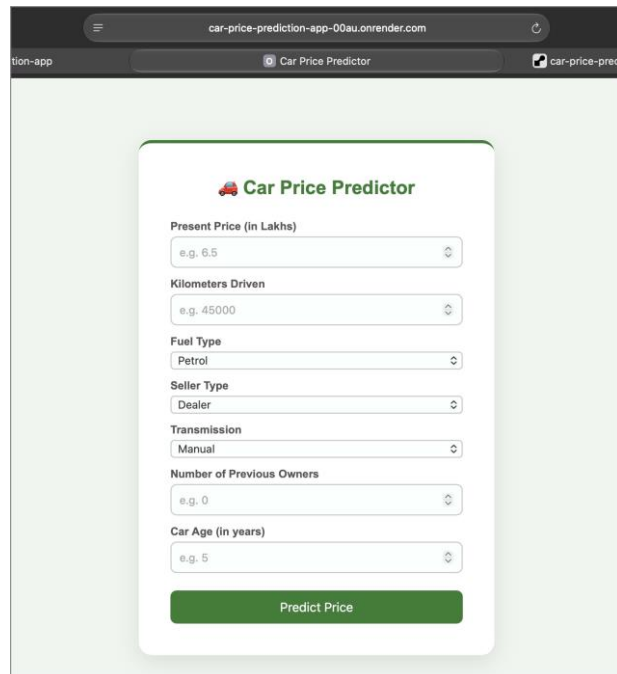
# ---- build and run locally ----
# docker build -t car-price-docker-app .
# docker run -p 10000:10000 car-price-docker-app
```

2.3.13 Output

Table 2.7 shows the estimates returned by the deployed model for three inputs. The first uses the example values shown as hints in the form; the other two are vehicles from the test set, so the estimate can be compared with the actual selling price. Figures 2.10 and 2.11 show the live application. After a prediction, the page shows the result below a fresh, empty form, so the input values behind Figure 2.11 are those in the first row of Table 2.7.

Table 2.7: Sample predictions of the deployed model

Input values	Estimated price	Actual price
Example from the form hints: present price 6.5 lakh, 45,000 km, Petrol, Dealer, Manual, 0 previous owners, 5 years old	₹4.43 lakh	Not applicable
Ertiga (2016, test set): present price 10.79 lakh, 43,000 km, Diesel, Dealer, Manual, 0 previous owners, 9 years old	₹7.90 lakh	₹7.75 lakh
City (2016, test set): present price 13.60 lakh, 30,753 km, Petrol, Dealer, Automatic, 0 previous owners, 9 years old	₹10.72 lakh	₹10.90 lakh



The screenshot shows a web browser displaying the 'Car Price Predictor' application. The browser's address bar shows the URL 'car-price-prediction-app-00au.onrender.com'. The application interface is centered on the page and features a green header with a car icon and the title 'Car Price Predictor'. Below the header, there are several input fields with placeholder text: 'Present Price (in Lakhs)' (e.g. 6.5), 'Kilometers Driven' (e.g. 45000), 'Fuel Type' (Petrol), 'Seller Type' (Dealer), 'Transmission' (Manual), 'Number of Previous Owners' (e.g. 0), and 'Car Age (in years)' (e.g. 5). Each input field has a small circular icon on the right side. At the bottom of the form is a green button labeled 'Predict Price'.

Figure 2.10: Home page of the live Car Price Prediction System

The screenshot shows a web browser window with the URL 'car-price-prediction-app-00au.onrender.com'. The page title is 'Car Price Predictor'. The main content is a form titled 'Car Price Predictor' with a car icon. The form contains the following fields:

- Present Price (in Lakhs): Input field with 'e.g. 6.5' and a dropdown arrow.
- Kilometers Driven: Input field with 'e.g. 45000' and a dropdown arrow.
- Fuel Type: Input field with 'Petrol' and a dropdown arrow.
- Seller Type: Input field with 'Dealer' and a dropdown arrow.
- Transmission: Input field with 'Manual' and a dropdown arrow.
- Number of Previous Owners: Input field with 'e.g. 0' and a dropdown arrow.
- Car Age (in years): Input field with 'e.g. 5' and a dropdown arrow.

Below the input fields is a green button labeled 'Predict Price'. Below the button is a green box displaying the 'Estimated Car Price: ₹ 4.43 Lakhs'.

Figure 2.11: Prediction returned by the live application for the example input (₹4.43 lakh)

2.3.14 Limitations and Future Enhancements

The project achieves its objectives, but the evaluation also revealed limitations that should be understood when using it:

- **Small data set:** 301 records, of which 101 are two-wheelers, with only 2 CNG vehicles and 11 vehicles with previous owners, so estimates for such inputs are less reliable.
- **Dependence on one input:** Present_Price has an importance of 0.885, so an inaccurate present price entered by the user directly distorts the estimate.
- **Age entered manually:** the model was trained with $\text{Car_Age} = 2025 - \text{Year}$, and the form asks for the age directly, so the user must calculate it.
- **Brand not used:** Car_Name was dropped, although the brand and model influence resale value.
- **No range checks:** the form accepts negative or unrealistic values, which the model still processes.
- **No extrapolation:** a random forest cannot predict prices above the range seen in training, so estimates for very expensive vehicles are capped.

These limitations point directly to the following enhancements:

- Train on a larger and more recent used-car data set with thousands of listings, separate cars from two-wheelers, and include the brand as a feature.
- Tune the hyperparameters (number of trees, maximum depth, minimum samples per leaf) with GridSearchCV and cross-validation, and compare gradient-boosting models.
- Build a scikit-learn Pipeline that performs encoding and feature creation inside the model, so that the form can accept the year of manufacture instead of the age.
- Add validation of minimum and maximum values in both the HTML form and the Flask route.
- Add a JSON API endpoint for other applications, and automated tests that run through GitHub Actions before each deployment.

2.4 Graded Assignments

The ten graded assignments assessed each major module with a practical task on a real data set. Each was implemented in a Jupyter notebook (or, for Assignment 10, as a complete application), documented with a README and submitted as a public GitHub repository named MPONLINE-Assignment-1 to MPONLINE-Assignment-10. Table 2.8 summarises them; the subsections that follow describe each assignment, its result and what was learned from it, including the problems found when the results were examined critically.

Table 2.8: Summary of the ten graded assignments

No.	Assignment	Technique	Data set (records)	Headline result
1	Medical Insurance Cost Prediction	Multiple Linear Regression	Medical Cost Personal (1,338)	R^2 0.784, MAE 4,181
2	Customer Churn Prediction	Logistic Regression	Telco Customer Churn (7,043)	Accuracy 81.97%, F1 0.636
3	Salary Prediction	Polynomial Regression (degree 3)	Position Salaries (10)	R^2 0.876 on 2 test points
4	Breast Cancer Classification	K-Nearest Neighbors ($k = 5$)	Breast Cancer Wisconsin (569)	Accuracy 95.61%, recall 90.48%
5	Employee Attrition Prediction	Decision Tree vs Random Forest	IBM HR Attrition (1,470)	Accuracy 76.53% vs 83.33%

No.	Assignment	Technique	Data set (records)	Headline result
6	Weather Condition Classification	SVM (RBF kernel)	Open-Meteo, New Delhi (4,824)	Accuracy 98.86% (see 2.4.6)
7	Customer Segmentation	K-Means + PCA	Mall Customers (200)	5 clusters; 59.92% variance in 2 PCs
8	Handwritten Digit Recognition	ANN (TensorFlow/Keras)	MNIST in CSV (60,000)	Accuracy 97.41%
9	Cats vs Dogs Classification	CNN (TensorFlow/Keras)	Cats and dogs images (24,998)	Accuracy 85.20%
10	Heart Disease Prediction API	Random Forest + Flask + Render	Heart Disease (1,025)	Live REST API (see 2.4.10)

2.4.1 Assignment 1: Medical Insurance Cost Prediction using Multiple Linear Regression

The objective was to predict annual medical insurance charges from personal attributes. The Medical Cost Personal data set contains 1,338 records with age, sex, body-mass index (BMI), number of children, smoking status and region, with the charges as the target. After checking for missing values, the categorical columns were one-hot encoded with `drop_first`, and the data was split 80/20 into 1,070 training and 268 test records. A `LinearRegression` model achieved an **R^2 of 0.7836**, an **MAE of 4,181.19** and an MSE of 33,596,915.85 (an RMSE of about 5,796) on the test set, so it explains about 78% of the variation in the charges.

Figure 2.12 shows that the predictions are accurate for low and medium charges but spread widely for the highest charges. This is a known property of the data: charges rise sharply when smoking is combined with a high BMI, and a purely additive linear model cannot represent such an interaction. Adding an interaction feature ($\text{smoker} \times \text{BMI}$) or using a tree-based model would be the natural next step.

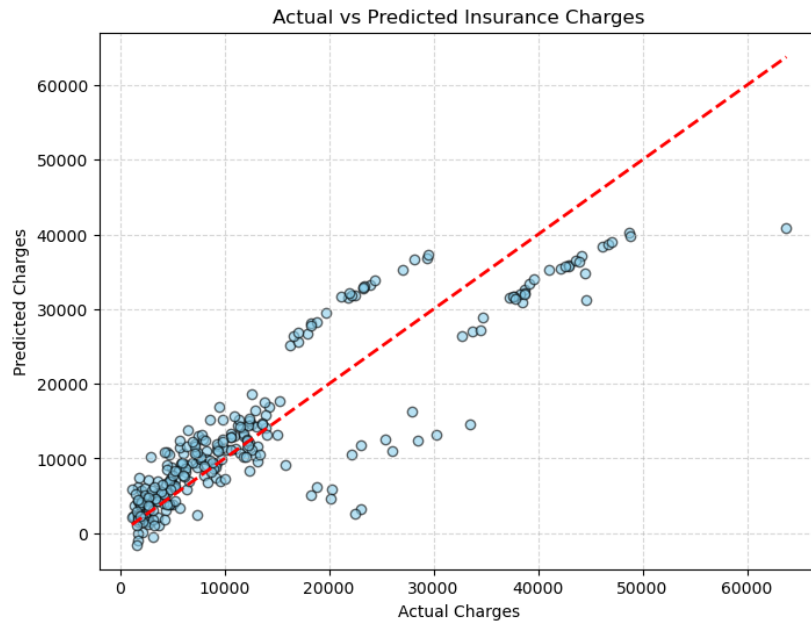


Figure 2.12: Assignment 1: actual versus predicted insurance charges

2.4.2 Assignment 2: Customer Churn Prediction using Logistic Regression

The Telco Customer Churn data set contains 7,043 customers with account, service and billing information and a Churn label. The TotalCharges column is stored as text and contains blanks for new customers; it was converted to numbers and the blanks were filled with the median. The customerID column was dropped, the categorical features were one-hot encoded (30 input features in total), the data was split 80/20 into 5,634 training and 1,409 test customers, and the features were standardised. A LogisticRegression model (`max_iter = 1000`) achieved an **accuracy of 81.97%**, a precision of 68.31%, a recall of 59.52% and an **F1-score of 0.6361**. The confusion matrix (Figure 2.13) shows 933 true negatives, 103 false positives, 151 false negatives and 222 true positives.

Two observations put these numbers in context. First, 73.5% of the test customers did not churn, so a model that always predicts "no churn" would already reach 73.5% accuracy; the real gain from the model is therefore smaller than the accuracy suggests. Second, 151 of the 373 churners were missed. Since a retention team wants to find as many likely churners as possible, lowering the decision threshold or using `class_weight = "balanced"` would raise recall at the cost of more false alarms.

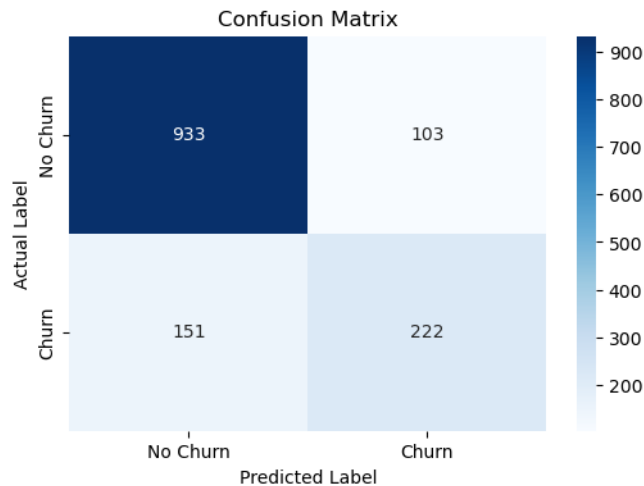


Figure 2.13: Assignment 2: confusion matrix of the churn model

2.4.3 Assignment 3: Salary Prediction using Polynomial Regression

The Position Salaries data set contains only 10 rows: position levels 1 to 10 and the corresponding salaries, which rise from 45,000 to 1,000,000. A degree-3 polynomial regression model was built with `PolynomialFeatures` and `LinearRegression` after an 80/20 split, which left 8 points for training and 2 for testing (levels 9 and 2). On these two points the model achieved an **MAE of 70,635.25**, an MSE of 6,263,853,282.86 and an **R^2 of 0.8763**. Figure 2.14 shows the fitted curve.

With only two test points, however, the score depends heavily on which points happen to be held out: repeating the split with 100 different random seeds gave R^2 values ranging from large negative values up to 0.92, with a median of 0.74. A more reliable evaluation for such a small data set is leave-one-out cross-validation (LOOCV), in which each of the 10 points is predicted by a model trained on the other nine. LOOCV gave R^2 values of 0.34, 0.67, 0.84 and 0.93 for degrees 1 to 4, confirming that a curved model is needed and that degree 3 or 4 fits this data well. The fitted curve also dips slightly between levels 3 and 5, showing that a flexible polynomial fitted on few points can behave unrealistically between them.

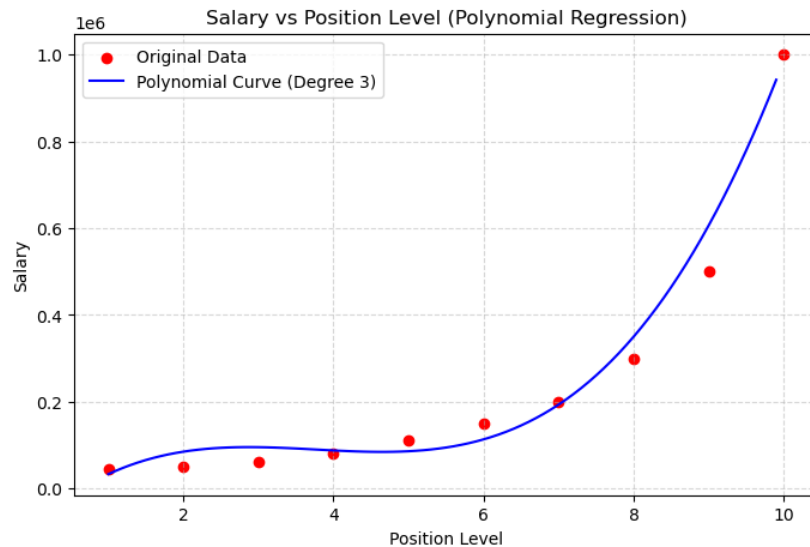


Figure 2.14: Assignment 3: degree-3 polynomial fitted to the salary data

2.4.4 Assignment 4: Breast Cancer Classification using K-Nearest Neighbors

The Breast Cancer Wisconsin (Diagnostic) data set contains 569 tumour samples described by 30 numerical features computed from images of cell nuclei, each labelled malignant or benign. The id column and an empty trailing column were removed, the diagnosis was encoded (malignant = 1, benign = 0), and the data was split with stratification into 455 training and 114 test samples. Because KNN relies on distances, the features were standardised before training a `KNeighborsClassifier` with $k = 5$. The model achieved an **accuracy of 95.61%**, a precision of 97.44%, a **recall of 90.48%** and an F1-score of 0.9383. The confusion matrix (Figure 2.15) shows 71 of 72 benign and 38 of 42 malignant tumours classified correctly, with one false alarm and four missed malignant cases. Five-fold cross-validation on the training data gave about 96.5% accuracy for $k = 5$, so the result is stable.

In medical screening, a missed malignant tumour (a false negative) is far more costly than a false alarm. Classifying a sample as malignant when at least one of its five nearest neighbours is malignant (a probability threshold of 0.2 instead of 0.5) reduces the missed cases from four to one and raises recall to 97.6%, at the cost of six false alarms instead of one. Choosing the threshold according to the cost of each type of error is therefore as important as choosing the algorithm.

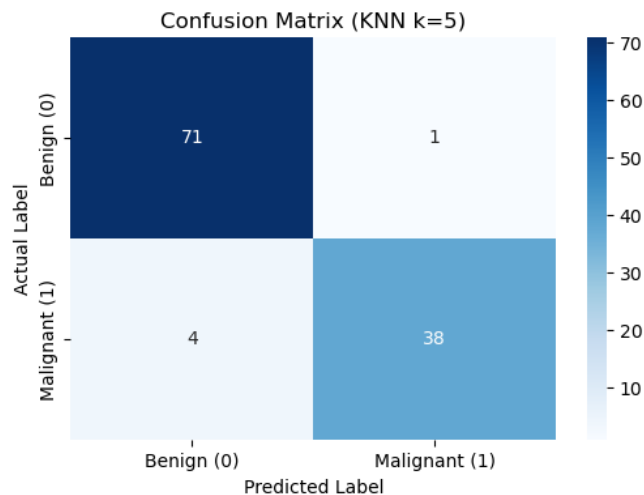


Figure 2.15: Assignment 4: confusion matrix of the KNN classifier ($k = 5$)

2.4.5 Assignment 5: Employee Attrition Prediction using Decision Tree and Random Forest

The IBM HR Analytics Employee Attrition data set contains 1,470 employees with 35 columns, such as age, monthly income, job role, overtime and years at the company, including the Attrition label. Identifier and constant columns were removed, the categorical columns were one-hot encoded (44 input features), and the data was split with stratification into 1,176 training and 294 test employees. A Decision Tree and a Random Forest were trained and compared (Figure 2.16). The decision tree achieved an accuracy of 76.53%, a precision of 31.03%, a recall of 38.30% and an F1-score of 0.3429; the random forest achieved an **accuracy of 83.33%**, a precision of 41.67%, a **recall of 10.64%** and an F1-score of 0.1695. The random forest's most important features were MonthlyIncome, Age and TotalWorkingYears (Figure 2.17).

The results illustrate the class-imbalance problem. Only 47 of the 294 test employees left the company, so a model that always predicts "stays" would reach 84.0% accuracy, which is higher than both models. The random forest's higher accuracy comes from predicting "stays" almost every time: it identified only 5 of the 47 leavers, whereas the less accurate decision tree identified 18. For this problem, recall and the F1-score of the leaving class are the right measures, and techniques such as `class_weight = "balanced"`, a limited tree depth and threshold tuning should be applied.

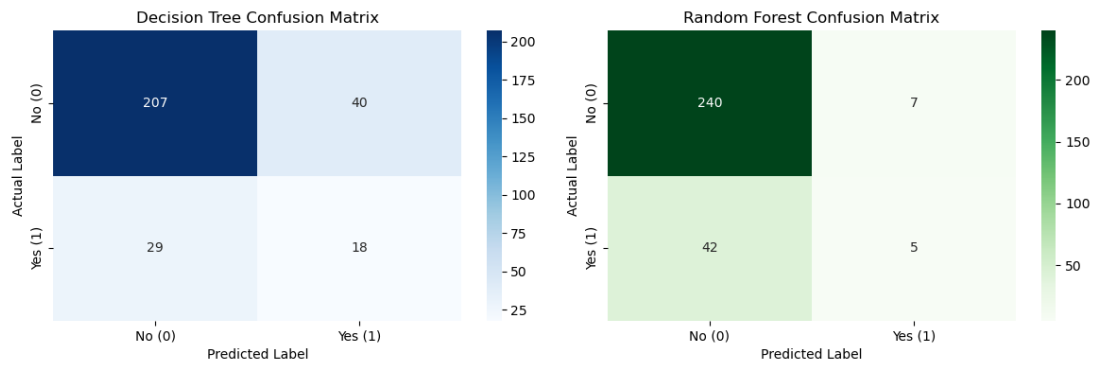


Figure 2.16: Assignment 5: confusion matrices of the Decision Tree (left) and the Random Forest (right)

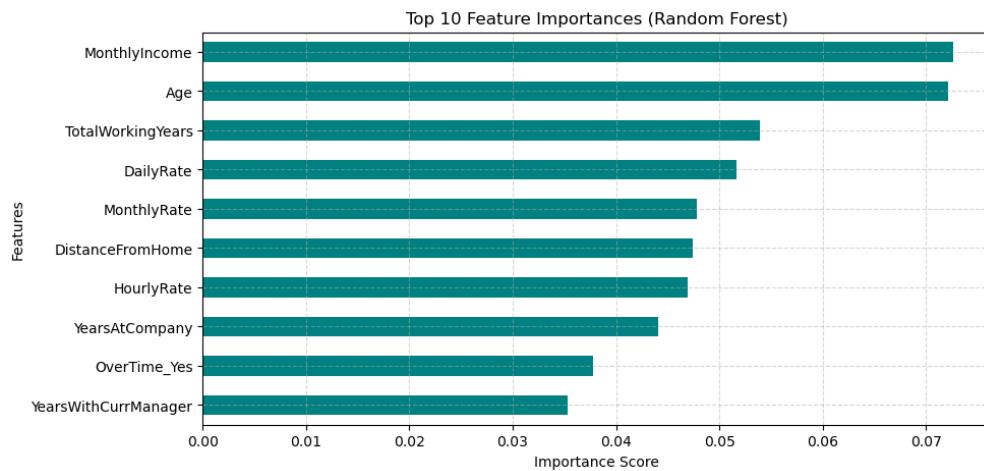


Figure 2.17: Assignment 5: top ten feature importances of the Random Forest

2.4.6 Assignment 6: Weather Condition Classification using Support Vector Machine

This assignment required collecting real data from an API. Hourly weather data for New Delhi (latitude 28.61, longitude 77.23) from 1st January to 20th July, 2026 was downloaded from the Open-Meteo historical weather API [31], giving 4,824 hourly records of temperature, relative humidity, surface pressure and wind speed. Each hour was labelled **Warm** (temperature of 25 °C or more) or **Cool**, giving 2,930 warm and 1,894 cool records. The time column was dropped, the data was split with stratification into 3,859 training and 965 test records, the features were standardised, and an SVM with an RBF kernel was trained. It achieved an **accuracy of 98.86%**, a precision of 99.15%, a recall of 98.98% and an F1-score of 0.9906, with only 11 misclassified hours (Figure 2.18).

The very high score has a simple explanation: the label was created from the temperature_2m column, and the same column was also used as an input feature. The

model therefore only had to learn the threshold of 25 °C, which is a form of **data leakage**. A meaningful version of this task removes the temperature from the inputs and predicts the class from humidity, pressure, wind speed and time features such as the hour and the month, using a chronological split so that the model is tested on dates later than those it was trained on.

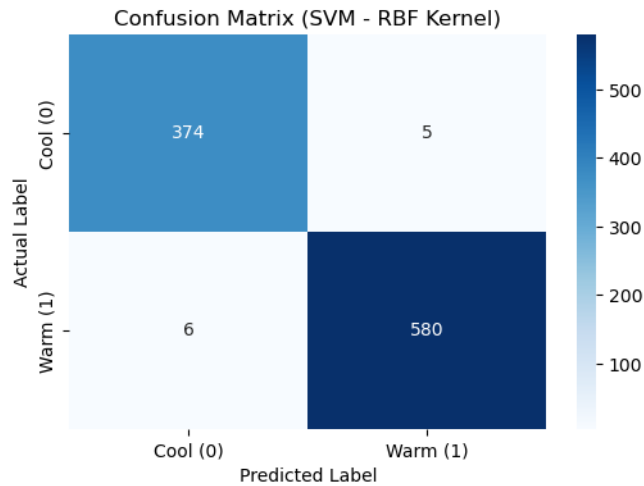


Figure 2.18: Assignment 6: confusion matrix of the SVM (RBF kernel)

2.4.7 Assignment 7: Customer Segmentation using K-Means Clustering and PCA

The Mall Customers data set contains 200 customers with gender, age, annual income (in thousands of dollars) and a spending score from 1 to 100. CustomerID was removed, gender was encoded as a number and the four features were standardised. The elbow method was used to choose the number of clusters, K-Means was trained with **K = 5**, and PCA reduced the data to two components for visualisation (Figure 2.19); these two components explain **59.92%** of the variance (33.69% and 26.23%). Table 2.9 profiles the five clusters.

Table 2.9: Assignment 7: profile of the five customer clusters

Cluster	Customers	Male (%)	Mean age	Mean income (k\$)	Mean spending score
0	39	49	33	86.5	82
1	29	41	37	89.5	18
2	43	0	50	49.2	40
3	54	43	25	39.7	61
4	35	100	56	53.7	37

Some segments are directly useful for marketing: cluster 0 contains young, high-income, high-spending customers, and cluster 1 contains high-income customers who spend little. However, the WCSS curve (800, 589, 477, 389, 331, 276 and 236 for $K = 1$ to 7) has no sharp elbow, and including gender as a feature produced two clusters defined largely by gender (clusters 2 and 4). Clustering on annual income and spending score alone yields the five well-known customer segments with a much higher silhouette score (0.555, compared with 0.304 for the four-feature model), which shows that feature selection matters as much in unsupervised learning as in supervised learning.



Figure 2.19: Assignment 7: elbow curve (left), clusters by income and spending (centre) and the clusters in PCA space (right)

2.4.8 Assignment 8: Handwritten Digit Recognition using an Artificial Neural Network

The MNIST data set in CSV form provides 60,000 labelled 28×28 grayscale images of handwritten digits [16], each stored as 784 pixel values. The pixels were scaled to the range 0 to 1, the labels were one-hot encoded, and the data was split with stratification into 48,000 training and 12,000 test images. A Keras Sequential network with two hidden layers ($784 \rightarrow 128 \text{ ReLU} \rightarrow 64 \text{ ReLU} \rightarrow 10 \text{ softmax}$, 109,386 trainable parameters) was trained with the Adam optimiser and categorical cross-entropy for 10 epochs with a batch size of 64, holding out 10% of the training data for validation. The network reached a **test accuracy of 97.41%** with a loss of 0.0983 and a macro-averaged F1-score of 0.9739.

The training curves (Figure 2.20) show that the validation loss was lowest at epoch 6 (0.096) and rose to 0.121 by epoch 10 while the training accuracy kept rising to 99.4%, the classic sign of the onset of overfitting. An EarlyStopping callback that restores the best weights would stop training at that point and keep the better model.

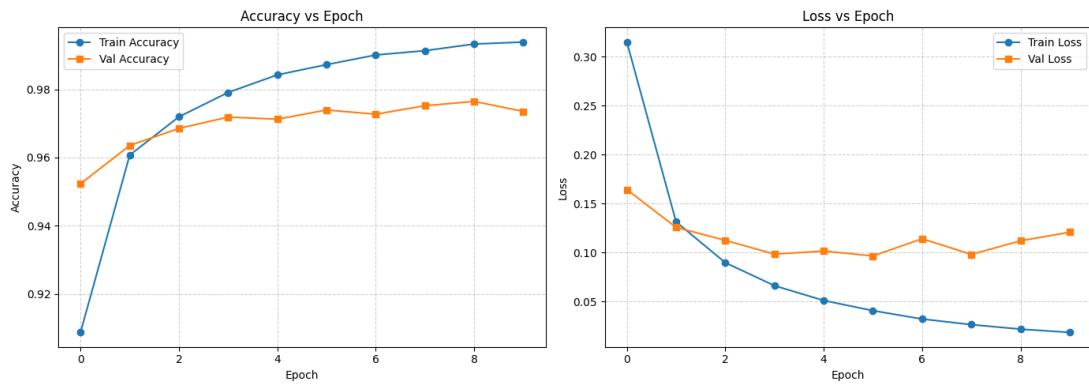


Figure 2.20: Assignment 8: training and validation accuracy (left) and loss (right) of the ANN

2.4.9 Assignment 9: Cats vs Dogs Image Classification using a Convolutional Neural Network

A CNN was built to classify photographs of cats and dogs from a Kaggle data set of 24,998 images (12,499 of each class), resized to 128×128 pixels and split with stratification into 19,998 training and 5,000 test images. The network has three convolutional blocks with 32, 64 and 128 filters, each followed by max pooling, and a dense layer of 128 neurons with a sigmoid output; it was trained for 10 epochs with the Adam optimiser and binary cross-entropy. The model reached a **test accuracy of 85.20%**, a precision of 84.43%, a recall of 86.32% and an F1-score of 85.36%, and the confusion matrix (Figure 2.21) shows 2,102 cats and 2,158 dogs classified correctly.

Two lessons emerged. First, the training accuracy reached 98.9%, while the validation loss was lowest at epoch 3 (0.346) and rose to 0.719 by epoch 10 (Figure 2.22), so the network overfitted strongly. About 97% of its 3,304,769 parameters sit in the dense layer after flattening; data augmentation, dropout, global average pooling and early stopping would reduce this. Second, the test images were also passed to training as validation data, so they influenced how training was monitored; a separate validation split should be used so that the test set remains untouched until the final evaluation.

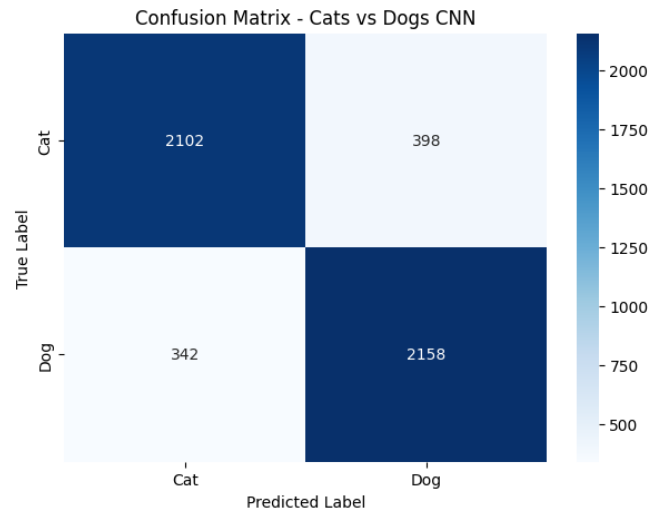


Figure 2.21: Assignment 9: confusion matrix of the CNN on 5,000 test images

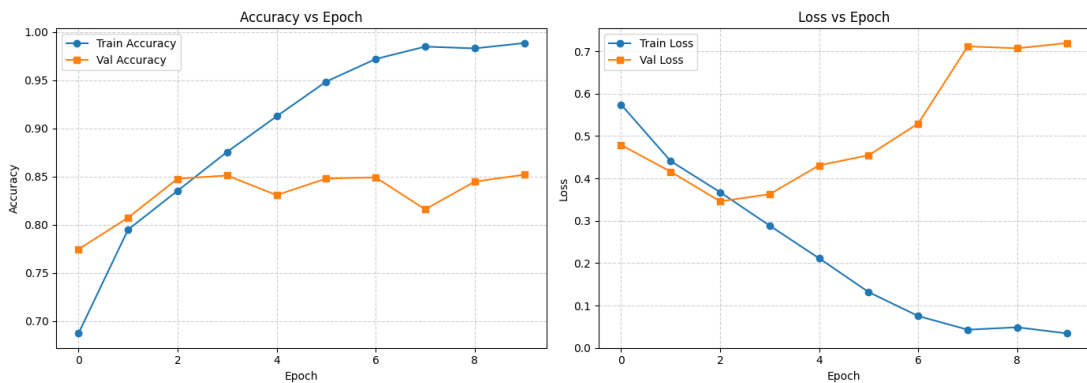


Figure 2.22: Assignment 9: training and validation accuracy (left) and loss (right) of the CNN

2.4.10 Assignment 10: End-to-End Heart Disease Prediction API

The final assignment required a complete, deployed machine learning service. The Heart Disease data set contains 1,025 patient records with 13 clinical features, such as age, sex, chest-pain type, resting blood pressure, cholesterol, maximum heart rate and ST depression, and a binary target. A Random Forest classifier with 100 trees was trained on an 80/20 stratified split and saved with Joblib as model.pkl. A Flask REST API was then built with two routes: GET / returns a JSON status message, and POST /predict accepts the 13 features as JSON and returns the prediction ("Heart Disease Detected" or "No Heart Disease Detected") together with a numeric prediction code (Listing 2.7). The code was pushed to the repository MPONLINE-Assignment-10 and deployed on Render with Gunicorn at <https://mponline-assignment-10-zqlt.onrender.com> (Figure 2.23).

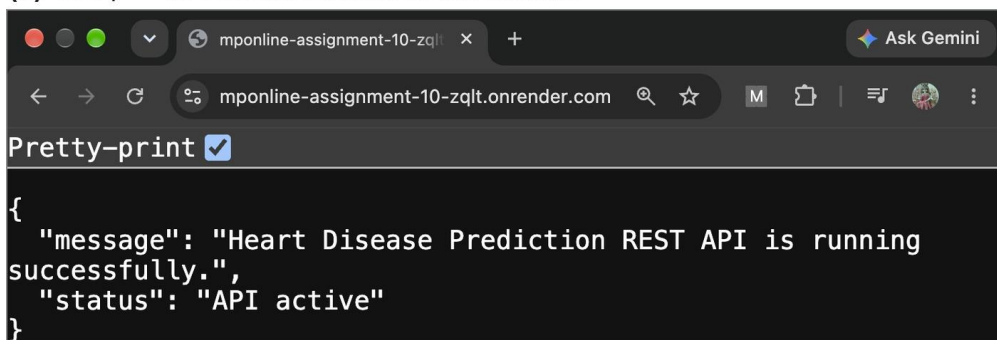
Listing 2.7: Calling the deployed heart disease API and its JSON response

```
curl -X POST https://mponline-assignment-10-zqlt.onrender.com/predict \
-H "Content-Type: application/json" \
-d '{"age": 52, "sex": 1, "cp": 0, "trestbps": 125, "chol": 212, "fbs": 0,
    "restecg": 1, "thalach": 168, "exang": 0, "oldpeak": 1.0,
    "slope": 2, "ca": 2, "thal": 3}'

# response returned by the API
{"prediction": "No Heart Disease Detected", "prediction_code": 0}
```

The model reported an accuracy of 100% on the test set. Investigating this result showed that the file contains 723 duplicate rows: only 302 records are unique, and 202 of the 205 test rows also appeared in the training data, so the model was largely recognising patients it had already seen. After removing the duplicates, the same model achieved 75.4% accuracy on a 61-patient test set and 82.8% in 5-fold cross-validation, which are the realistic figures. Two deployment problems were also solved: port 5000 is occupied on macOS by the AirPlay Receiver, so the API was run locally on port 5001, and Render's default Python version (3.14) differed from the local version (3.10), so the Python version was pinned explicitly (Section 2.8).

(a) GET / — API status checked in the browser



(b) POST /predict — prediction requested from the terminal with curl

```
(base) akshatgarg@Akshats-MacBook-Pro ~ % curl -X POST https://mponline-assignment-10-zqlt.onrender.com/predict -H "Content-Type: application/json" -d '{"age": 52, "sex": 1, "cp": 0, "trestbps": 125, "chol": 212, "fbs": 0, "restecg": 1, "thalach": 168, "exang": 0, "oldpeak": 1.0, "slope": 2, "ca": 2, "thal": 3}'
{"prediction": "No Heart Disease Detected", "prediction_code": 0}
(base) akshatgarg@Akshats-MacBook-Pro ~ %
```

Figure 2.23: Assignment 10: the deployed heart disease prediction API, (a) status check with GET / in the browser and (b) a prediction from POST /predict using curl

2.5 Additional Projects

Besides the graded assignments, I built the following projects to practise the remaining modules in more depth. All of them are published on GitHub, and the links are collected in Section 2.9.

2.5.1 Reinforcement Learning: CartPole and Lunar Lander

CartPole. In CartPole-v1, the agent must keep a pole balanced on a moving cart by pushing the cart left or right, receiving a reward of +1 for every step the pole stays upright, up to a maximum of 500 steps. A PPO agent from Stable-Baselines3 with a multilayer-perceptron policy (MlpPolicy, learning rate 3×10^{-4} , 2,048 steps per update) was trained for 50,000 timesteps. The learning curve (Figure 2.24) rises steadily until the agent reaches the maximum reward. Evaluated deterministically over 100 episodes, the trained agent scored a **mean reward of 500.00 ± 0.00** , meaning that it balanced the pole for the full episode every time.

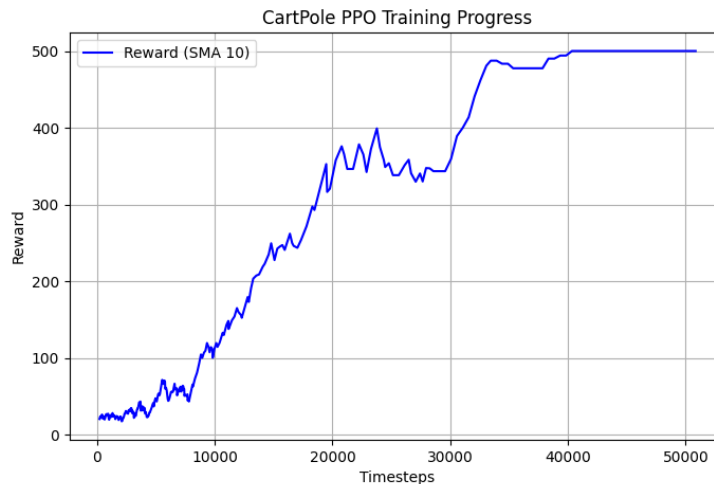


Figure 2.24: Learning curve of the PPO agent on CartPole-v1 (10-episode moving average)

Lunar Lander. LunarLander-v3 is considerably harder. The agent controls a lander with four discrete actions (do nothing, fire the left engine, fire the main engine, fire the right engine) based on an 8-dimensional observation (position, velocity, angle, angular velocity and two leg-contact flags). It is rewarded for landing gently on the pad between the flags and penalised for crashing and for using fuel, and a score of 200 or more is considered a solved episode. A PPO agent (learning rate 3×10^{-4} , gamma 0.99, batch size 64, 2,048 steps and 10 epochs per update, clip range 0.2) was trained for 500,000 timesteps, which took 1,364 episodes. The learning curve (Figure 2.25) rises from

strongly negative rewards, when the lander crashes, to around 150–250. Evaluated over 20 deterministic episodes, the agent scored a **mean reward of 170.91 ± 101.20** , which falls in the "good" band (100 to 200) of the evaluation scale in the course notes. Five separate test runs scored 256.16, 193.09, 233.42, 240.13 and 234.41, four of which exceed the solving threshold. The large standard deviation shows that some landings still fail; longer training or hyperparameter tuning would make the agent more consistent. Figure 2.26 shows frames from a recorded landing. Installing the Box2D physics engine on macOS required SWIG, which was installed from conda-forge.

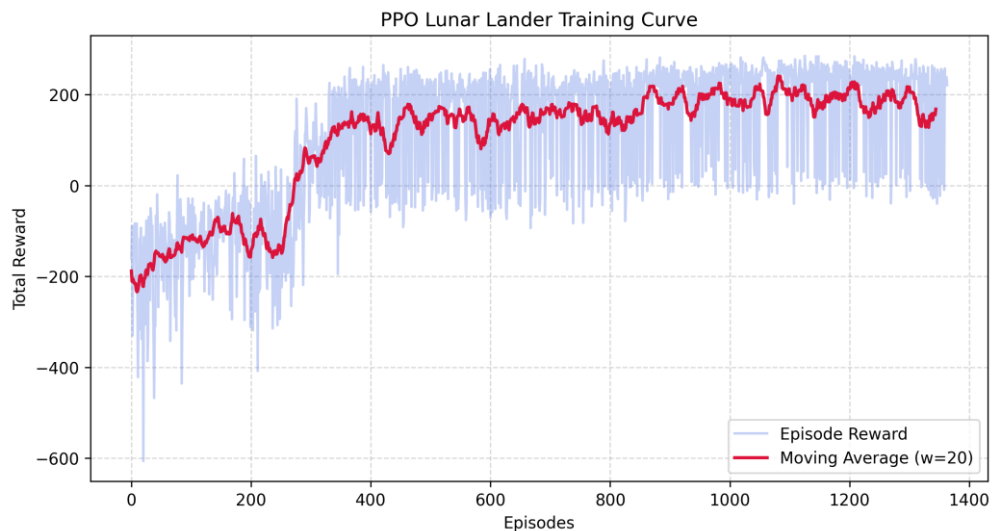


Figure 2.25: Learning curve of the PPO agent on LunarLander-v3 over 1,364 training episodes



Figure 2.26: Frames from the recorded evaluation episode: descent, approach and landing between the flags

2.5.2 Retrieval-Augmented Generation Chatbots

Two chatbots were built to apply the RAG sessions. Both follow the pipeline in Figure 2.27: documents are indexed once, and every question is answered by retrieving the most relevant chunks and passing them to the language model as context.

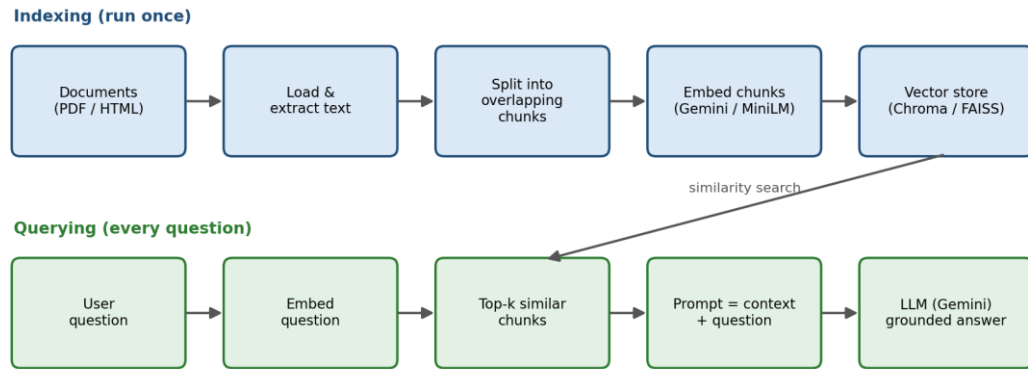


Figure 2.27: Retrieval-Augmented Generation pipeline used in both chatbots

Document Knowledge Base chatbot (repository RAG-Chatbot). An ingestion script loads all PDF files from a data folder with LangChain [29], splits them into chunks of 1,000 characters with an overlap of 150, embeds the chunks with Google's gemini-embedding-001 model and stores them in a persistent ChromaDB vector store. A Streamlit chat interface then retrieves the six most similar chunks for each question and sends them to Gemini 2.5 Flash (temperature 0.3) with a prompt that instructs the model to answer strictly from the provided context and to say so when the answer is not present. To stay within the free-tier limit on embedding requests, chunks are embedded in batches of 40 with a five-second pause between batches and an automatic retry after 35 seconds whenever the rate limit is exceeded. The 44-page sample document, a guide to working with machine learning data sets, was split into 144 chunks; Figure 2.28 shows the chatbot, run locally, summarising this guide in answer to a question.

Amazon quarterly report chatbot. This production-style version answers questions about Amazon's Form 10-Q for the quarter ended 31st March, 2026, downloaded from the SEC EDGAR system. It is organised into separate configuration, ingestion, retrieval and API modules. The filing text is split with a sliding-window chunker that prefers paragraph boundaries (1,200 characters with an overlap of 200), embedded locally with the all-MiniLM-L6-v2 sentence-transformer and indexed with FAISS. A FastAPI server retrieves the top five excerpts for each question, asks Gemini 2.5 Flash for a grounded answer and returns it with citations to the excerpts used, through a simple JavaScript chat page. A render.yaml blueprint describes the deployment, including a health-check endpoint and a CPU-only PyTorch build to keep the image small.

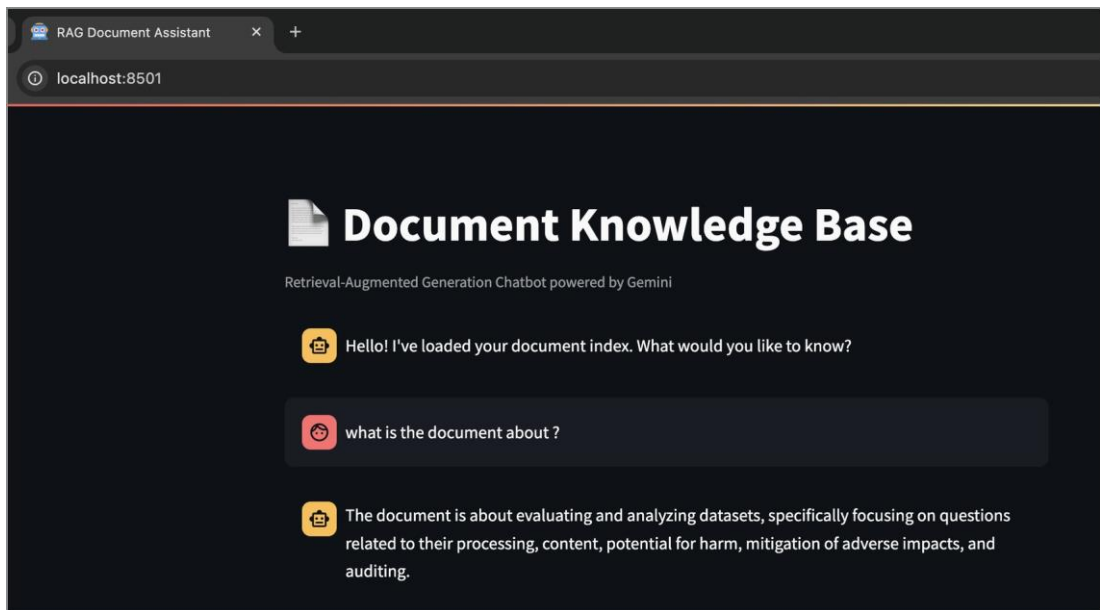


Figure 2.28: Document Knowledge Base RAG chatbot (Streamlit, running locally) answering a question about the sample guide

2.5.3 Movie Recommendation System

A content-based movie recommender was built on the MovieLens "latest small" data set [15], [32], which lists 9,742 movies with their genres. The genre strings were converted into TF-IDF vectors, and when a user selects a movie in the Flask web page, the cosine similarity between that movie and every other movie is computed and the five most similar titles are shown. The similarity is computed only for the selected movie rather than as a full $9,742 \times 9,742$ matrix, which keeps memory use low enough for Render's free instance, where the application is deployed at <https://movie-recommendation-system-oo9b.onrender.com>. Because the 9,742 movies share only 951 distinct genre combinations, many movies have identical similarity scores and the ranking among them is arbitrary. Figure 2.29 shows this clearly: *3 Idiots* (2009) is tagged Comedy, Drama and Romance, and all five recommendations carry exactly the same three genres; in fact, 275 other movies in the data set share this combination, so the five shown are effectively picked from 275 equally similar titles. Adding tags or plot keywords, or using the ratings file for collaborative filtering, would make the recommendations more personal.

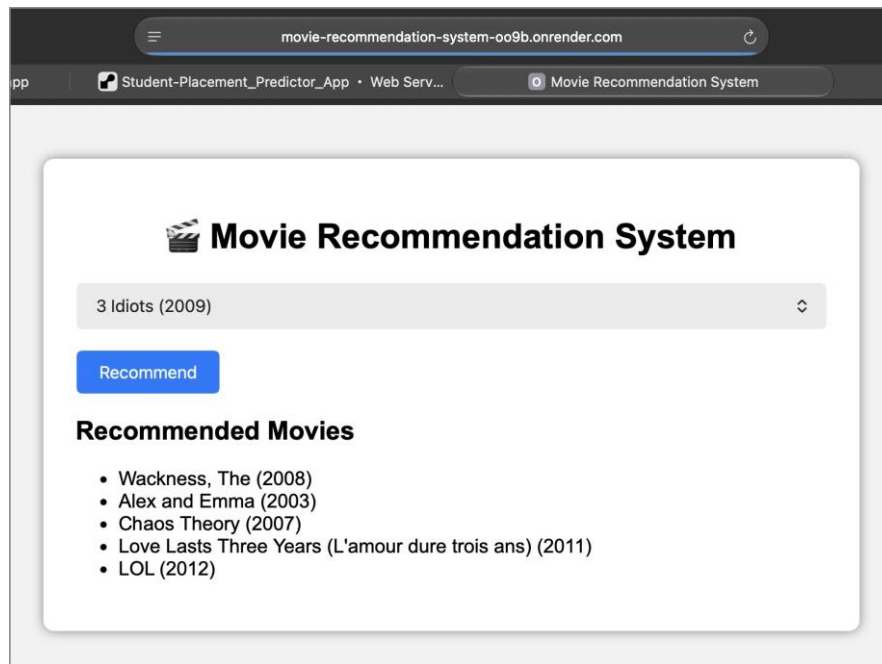


Figure 2.29: Movie Recommendation System deployed on Render, showing the recommendations for 3 Idiots (2009)

2.5.4 Student Placement Predictor

The Student Placement Predictor estimates whether a student is likely to be placed, using a Random Forest classifier trained on 2,000 student records (1,000 placed and 1,000 not placed) with five inputs: CGPA, communication skills, resume score, coding score and placement-training attendance. A Flask form collects the five values and displays the verdict ("Placed" or "Not Placed"). The application was containerised with a Dockerfile based on python:3.10-slim, served by Gunicorn on port 10000 and deployed as a Docker web service on Render at <https://student-placement-predictor-app-6rll.onrender.com>. Figure 2.30 shows the live application classifying a student with a CGPA of 8.1, communication and resume scores of 8 and 9, a coding score of 6 and 100% attendance as Placed.

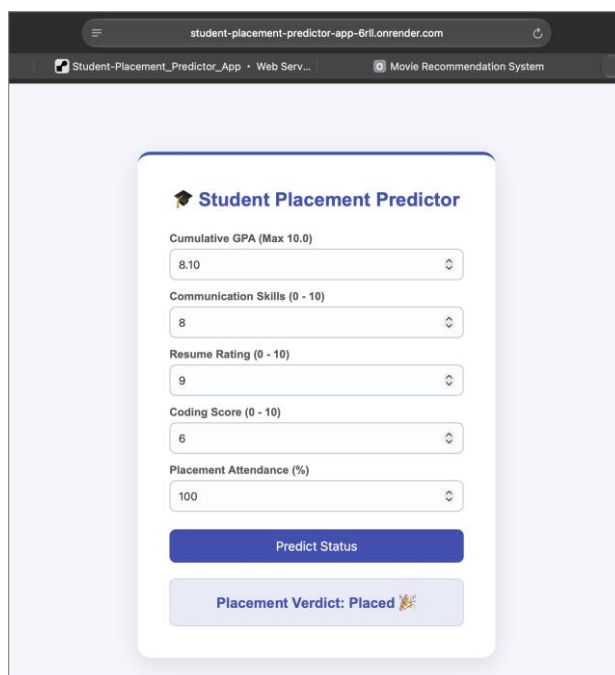


Figure 2.30: Student Placement Predictor deployed on Render with a sample prediction

2.5.5 Iris Flower Classifier API and Web Interface

This project followed the end-to-end deployment example of the MLOps week. A Random Forest classifier was trained on the Iris data set (150 flowers, 4 measurements, 3 species) and saved with Joblib. A Flask application serves a web page built with Tailwind CSS and a /predict endpoint that accepts the four measurements as JSON and returns the predicted species as JSON. The application reads the port from the PORT environment variable and is packaged in a Docker image based on python:3.12-slim, so it can run unchanged on a laptop or on a cloud platform.

2.5.6 Deep Learning Projects: CIFAR-10, Brain Tumour MRI and Face Recognition

Three further CNN projects applied the deep learning module to harder image problems. On **CIFAR-10** [17] (50,000 training and 10,000 test colour images of 32×32 pixels in 10 classes), a three-block CNN with batch normalisation, dropout, data augmentation (rotation, shifts and horizontal flips) and learning-rate reduction reached a **test accuracy of 86.04%** after 30 epochs; the confusion matrix (Figure 2.31) shows that the weakest class is cat (653 of 1,000 correct), most often confused with dog and frog, while most errors among vehicles are automobiles predicted as trucks. The **Brain Tumour MRI** project classified about 5,600 training and 1,600 test MRI scans into four classes (glioma, meningioma, pituitary tumour and no tumour) using 64×64

grayscale inputs and a CNN with global average pooling, reaching **91.75%** accuracy (Figure 2.32). The **LFW face recognition** project classified 1,288 face images of seven people into their identities with **97.67%** accuracy.

In the MRI and LFW projects, the test set was also used as the validation data for early stopping and for restoring the best weights, so the reported accuracies are slightly optimistic; a separate validation split would give an unbiased estimate. This observation is discussed further in Section 2.6.

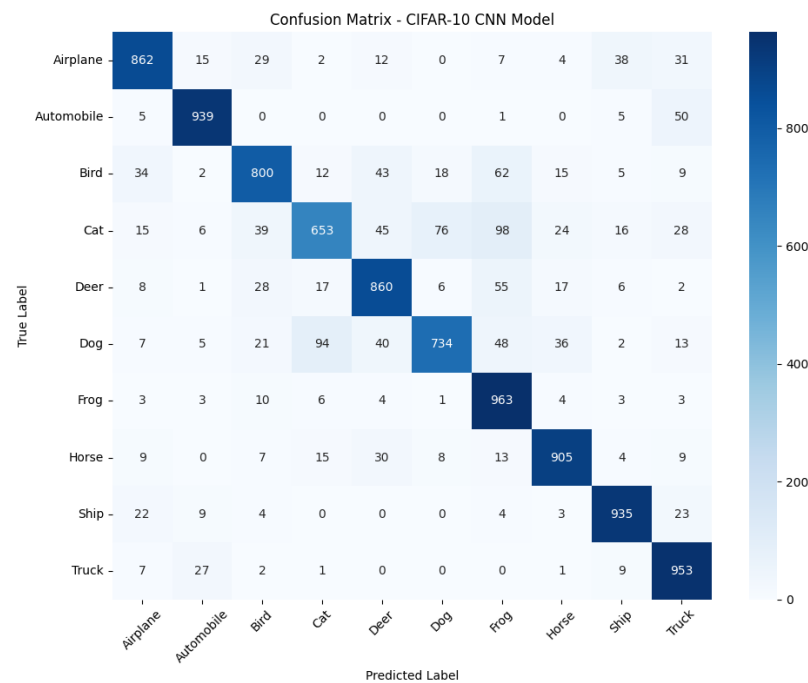


Figure 2.31: Confusion matrix of the CIFAR-10 CNN on 10,000 test images

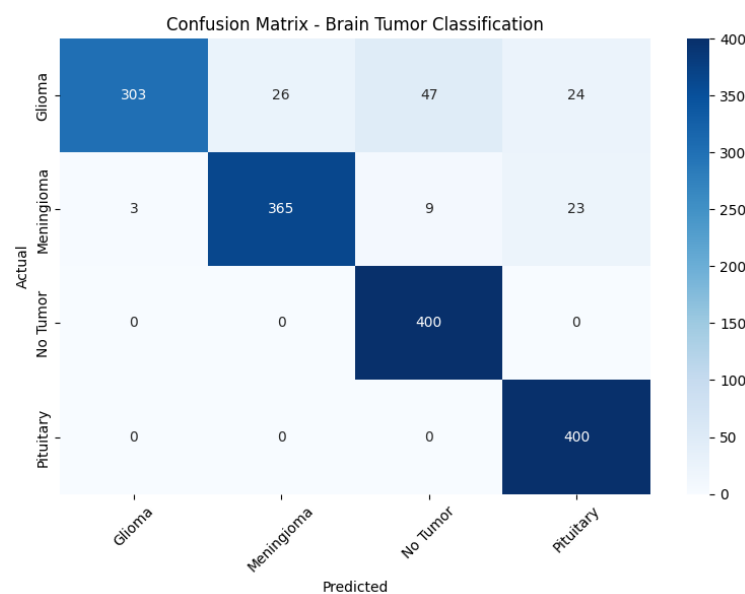


Figure 2.32: Confusion matrix of the brain tumour MRI classifier

2.5.7 Adult Census Income: Comparison of Five Classifiers

To compare the classical algorithms under identical conditions, five classifiers were trained on the Adult Census Income data set, which predicts whether a person earns more than \$50,000 a year from demographic and employment attributes. Missing values marked "?" were filled with the most frequent value, duplicate rows were removed (32,537 records remained), the categorical features were one-hot encoded (97 features), and the data was split with stratification and standardised. Table 2.10 shows the results on the test set.

Table 2.10: Comparison of five classifiers on the Adult Census Income data set

Algorithm	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.8505	0.7352	0.5931	0.6565	0.8988
Random Forest	0.8482	0.7217	0.6020	0.6565	0.8940
Support Vector Machine	0.8419	0.7259	0.5523	0.6273	0.8841
K-Nearest Neighbors	0.8208	0.6493	0.5574	0.5999	0.8260
Decision Tree	0.8059	0.5950	0.6091	0.6020	0.7387

Logistic regression and the random forest performed almost identically and best overall, showing that a simple, well-prepared linear model can be as good as an ensemble on data of this kind. The unpruned decision tree had the lowest ROC-AUC because it overfits its training data, and KNN suffered from the high number of one-hot encoded dimensions. All models found it harder to identify the high-income minority class (recall of about 0.55 to 0.61) than to achieve high overall accuracy, which again shows the importance of looking beyond accuracy.

2.6 Results and Observations

Table 2.11 brings together the main results of all the projects and assignments discussed in this chapter. The observations that follow summarise what these results, taken together, taught about building machine learning systems.

Table 2.11: Consolidated results of the internship work

Project	Technique	Measure	Result
Car Price Prediction (main project)	Random Forest Regressor	R^2 / RMSE (test set)	0.958 / 0.98 lakh

Project	Technique	Measure	Result
Medical insurance cost (A1)	Multiple Linear Regression	R^2 / MAE	0.784 / 4,181
Customer churn (A2)	Logistic Regression	Accuracy / F1-score	81.97% / 0.636
Salary prediction (A3)	Polynomial Regression	R^2 (2-point test / LOOCV)	0.876 / 0.837
Breast cancer (A4)	KNN (k = 5)	Accuracy / recall	95.61% / 90.48%
Employee attrition (A5)	Decision Tree / Random Forest	Accuracy (F1-score)	76.53% (0.343) / 83.33% (0.170)
Weather classification (A6)	SVM (RBF kernel)	Accuracy	98.86% (leakage)
Customer segmentation (A7)	K-Means + PCA	Variance in 2 PCs / silhouette	59.92% / 0.304
Handwritten digits (A8)	ANN	Accuracy	97.41%
Cats vs dogs (A9)	CNN	Accuracy	85.20%
Heart disease API (A10)	Random Forest + Flask + Render	Accuracy (reported / deduplicated)	100% / 75.4%
CartPole	PPO	Mean reward (100 episodes)	500.00 $\pm$ 0.00
Lunar Lander	PPO	Mean reward (20 episodes)	170.91 $\pm$ 101.20
CIFAR-10	CNN	Accuracy	86.04%
Brain tumour MRI	CNN	Accuracy	91.75%
LFW face recognition	CNN	Accuracy	97.67%
Adult Census Income	Logistic Regression (best of 5)	Accuracy / ROC-AUC	85.05% / 0.899

1. Good data preparation matters more than the choice of algorithm. On the car price data, the random forest clearly outperformed linear regression (R^2 0.958 against 0.847) because the relationships are non-linear. On the Adult Census data, however, a well-prepared logistic regression matched the random forest. In both cases, the largest gains came from the preparation of the data: engineered features such as the age of a vehicle, correct encoding and scaling, and the removal of irrelevant columns.

2. Accuracy alone can be misleading. When one class is rare, a model can reach high accuracy by ignoring it. The churn model's 81.97% accuracy must be compared with the 73.5% achieved by always predicting "no churn", and in the attrition task the random forest's 83.33% accuracy was below the 84.0% baseline because it found only 5 of the 47 leavers. Precision, recall, the F1-score and a comparison with a simple baseline give a truer picture.

3. Results that look too good usually point to a problem in the data. The heart disease model scored 100% because most test records were duplicates of training records, and the weather model scored nearly 99% because the label was derived from one of the input features. Checking for duplicates, checking how the target was created and asking whether any feature could reveal the answer are now the first things I do when a score looks surprisingly high.

4. The design of the evaluation matters as much as the model. A test set of two points (salary) or even 61 records (car price) gives a noisy estimate, and cross-validation gives a more dependable one. In the cats-versus-dogs, brain tumour and face recognition projects, the test set was also used during training for validation, so it was no longer an independent check; a separate validation split keeps the final test honest.

5. Neural networks overfit quickly without regularisation. In both the ANN and the first CNN, the validation loss began to rise after a few epochs while the training accuracy kept improving. The later CNN projects, which used data augmentation, dropout, batch normalisation, early stopping and learning-rate reduction, generalised much better.

6. Deployment depends on reproducibility. Every deployed project worked only after the Python version and library versions on the server matched those used to train and save the model. Pinning versions in requirements.txt, fixing the Python version and packaging with Docker turned deployment from trial and error into a repeatable process.

Table 2.12 records the corrected or cross-checked figures obtained when the results were re-examined for this report.

Table 2.12: Corrected and cross-checked results

Project and issue	Originally reported	Corrected or cross-checked result
Heart disease API (A10): duplicate records	Accuracy 100%	75.4% on a deduplicated test set; 82.8% with 5-fold cross-validation
Salary prediction (A3): two-point test set	R^2 0.876	LOOCV R^2 0.837 for degree 3 and 0.932 for degree 4
Breast cancer (A4): choice of threshold	Recall 90.48% (4 missed)	Recall 97.6% (1 missed) with a threshold of 0.2
Customer segmentation (A7): choice of features	Silhouette 0.304 (4 features)	Silhouette 0.555 using income and spending score only

Project and issue	Originally reported	Corrected or cross-checked result
Car price (main project): single test split	Test R^2 0.958	Mean 5-fold cross-validation R^2 0.920
Weather classification (A6): target leakage	Accuracy 98.86%	Not meaningful; the model must be retrained without the temperature feature (Section 2.4.6)

2.7 Skills Learned

2.7.1 Scientific and Technical Skills

- **Python for data science:** writing clean, modular Python and using NumPy and Pandas to load, clean, transform and summarise real data sets.
- **Machine learning with scikit-learn:** building, tuning and comparing regression, classification and clustering models, with preprocessing steps such as encoding and scaling.
- **Model evaluation and validation:** choosing metrics that match the problem, reading confusion matrices, using baselines, cross-validation and decision thresholds, and detecting leakage, duplicates and overfitting.
- **Deep learning with TensorFlow/Keras:** designing ANNs and CNNs, reading training curves, and applying augmentation, dropout, batch normalisation and early stopping.
- **Generative AI and RAG:** chunking and embedding documents, using vector databases (ChromaDB and FAISS), designing prompts for grounded answers and handling API rate limits.
- **Reinforcement learning:** formulating problems as environments with states, actions and rewards, and training and evaluating PPO agents with Stable-Baselines3.
- **Deployment and MLOps:** serialising models; building web applications and REST APIs with Flask and FastAPI; serving them with Gunicorn or Uvicorn; containerising with Docker; deploying on Render; and understanding experiment tracking with MLflow and CI/CD with GitHub Actions.
- **Data visualisation:** communicating findings with Matplotlib and Seaborn through correlation heatmaps, confusion matrices, learning curves and PCA scatter plots.

2.7.2 Professional Skills

- **Time management:** ten graded assignments were completed between 20th July and 1st August alongside new sessions, which required planning every day and starting early.
- **Working to specifications:** each assignment defined exact tasks, file names and submission formats, and reading the requirements carefully before starting became a habit.
- **Technical documentation:** every repository has a README that explains the objective, data set, method, results and how to run the project.
- **Version-control discipline:** Git and GitHub were used for every project, with .gitignore files for data and secrets and clear commit history.
- **Self-learning and problem solving:** installation, deployment and API problems were solved independently by reading official documentation and error messages.
- **Critical thinking:** results that looked too good were questioned and re-examined, and corrected figures are reported openly.
- **Security awareness:** API keys and credentials were kept out of code and repositories by using environment variables, .env files and prompts at run time.
- **Communication:** technical results were explained in plain language in READMEs, notebook observations and this report.

2.8 Challenges Faced and How They Were Overcome

Most of the challenges of the internship were practical rather than theoretical: installing libraries, deploying applications, working within the limits of free services and interpreting results correctly. Table 2.13 lists the main challenges, their causes and how they were resolved.

Table 2.13: Challenges faced during the internship and their resolution

Challenge	Cause	How it was resolved
Build failure when deploying to Render	Render's default Python version (3.14) was newer than the local version (3.10), and some pinned libraries were not available for it	Pinned the Python version of the service to match the local environment and kept all library versions pinned in requirements.txt

Challenge	Cause	How it was resolved
Local port conflict	Port 5000, the Flask default, is used by the AirPlay Receiver on macOS	Ran the API on port 5001 for local testing
Installing Box2D for Lunar Lander	The Box2D extra of Gymnasium compiles C++ code that requires SWIG	Installed SWIG from conda-forge before installing the Box2D dependency
API rate limits while embedding documents	The free tier of the Gemini API limits the number of requests per minute	Embedded chunks in batches of 40 with pauses between batches and an automatic retry after 35 seconds
Downloading Kaggle data in notebooks	The Kaggle API requires account credentials	Entered the username and key at run time, never storing them in the notebook, and used opendatasets or the Kaggle API
Large deployment size of the RAG chatbot	sentence-transformers installs PyTorch with multi-gigabyte GPU libraries by default	Installed the CPU-only PyTorch build through an additional package index in requirements.txt
Keeping repositories clean and secure	Data sets, environments, caches and .env files should not be committed	Added .gitignore files and committed only the small model files needed by the deployed applications
Imbalanced classes	The churn and attrition data contain few positive cases	Evaluated with precision, recall and F1-score against a majority-class baseline; identified class weighting and threshold tuning as remedies
Overfitting of neural networks	Large dense layers and many epochs without regularisation	Used augmentation, dropout, batch normalisation, early stopping and learning-rate reduction in the later CNN projects
Scores that looked too good	Duplicate records (heart disease) and a feature derived from the target (weather)	Checked for duplicates and target-derived features, and re-evaluated after deduplication and with cross-validation
Heavy workload in the final two weeks	Ten assignments due between 20th July and 1st August alongside new modules	Planned the work daily, reused a standard project structure (notebook, README, .gitignore) and submitted each assignment as soon as it was complete

The most valuable lesson from these challenges was that reliable machine learning depends on two habits: making every environment reproducible, by pinning versions and packaging applications, and verifying every result, by checking the data, the evaluation design and simple baselines before trusting a number.

2.9 Project Repositories and Live Links

All the work described in this report is publicly available on GitHub under the account **AkshatGarg2005** (<https://github.com/AkshatGarg2005>). Table 2.14 lists the repositories and the live deployments. The live services run on Render and may take up to a minute to respond to the first request after a period of inactivity.

Table 2.14: GitHub repositories and live deployments

Project	Repository (github.com/AkshatGarg2005/...)	Live deployment
Car Price Prediction System (main project)	car-price-prediction-app	car-price-prediction-app-00au.onrender.com
Car Price Prediction (Docker version)	car-price-predictor-app-docker	Runs as a Docker container
Student Placement Predictor	Student-Placement_Predictor_App	student-placement-predictor-app-6rll.onrender.com
Iris Flower Classifier API and Web UI	Iris-Flower-Classfier-API-Web-UI	Runs as a Docker container
CartPole PPO agent	CartPole	Runs locally
Lunar Lander PPO agent	Autonomous-Lunar-Landing-using-Proximal-Policy-Optimization	Runs locally
Movie Recommendation System	Movie-Recommendation-System	movie-recommendation-system-oo9b.onrender.com
Document Knowledge Base RAG chatbot	RAG-Chatbot	Runs locally (Streamlit)
Graded Assignments 1–9	MPONLINE-Assignment-1 to MPONLINE-Assignment-9	Jupyter notebooks
Assignment 10: Heart Disease Prediction API	MPONLINE-Assignment-10	mponline-assignment-10-zqlt.onrender.com
Portfolio: car price notebook, CIFAR-10, Brain Tumour MRI, LFW, Adult Census and Amazon 10-Q RAG chatbot	MPONLINE-Assignment-and-Project-Submission-Record	Notebooks; RAG chatbot runs locally

3. CONCLUSION

The AI/ML Internship Program at MPOnline Limited, carried out from 20th May, 2026 to 31st July, 2026, gave me a complete and practical grounding in artificial intelligence and machine learning. Over 53 working days the program covered Python and data analysis, the mathematics of ML, supervised and unsupervised learning, deep learning, NLP and Retrieval-Augmented Generation, recommendation systems, reinforcement learning, and deployment with Flask, Docker, GitHub and Render. All the original objectives were achieved, as shown by the ten graded assignments and the additional projects published in 19 public GitHub repositories.

The main project, the Car Price Prediction System, brought these skills together. Its Random Forest model predicts the resale price of a vehicle with an R^2 of 0.958 on unseen data and a cross-validated R^2 of about 0.92, and it is served to anyone through a Flask web application on Render, also packaged with Docker. Building it end to end showed that deployment is an integral part of machine learning: a model is useful only when it can be reproduced, served reliably and used by people who never see the code.

The most important lesson was about evaluation. Results that looked excellent at first, such as 100% accuracy on the heart disease data and almost 99% on the weather data, were caused by duplicate records and by a feature that leaked the answer, while other results were affected by class imbalance, very small test sets or test data reused for validation. Learning to question such results, to compare them with baselines and to use cross-validation has changed the way I approach every model.

Professionally, the internship developed time management under firm deadlines, clear technical documentation, disciplined version control and the confidence to solve installation and deployment problems independently. These skills complement the B.Tech CSE curriculum at VIT Bhopal University and will be useful in the rest of my degree and my career. As next steps, I plan to extend the main project with a larger data set, hyperparameter tuning and automated testing, and to study MLOps tools such as MLflow further.

4. BIBLIOGRAPHY

4.1 Books and Research Papers

- [1] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.
- [2] W. McKinney, *Python for Data Analysis*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.
- [3] G. James, D. Witten, T. Hastie, R. Tibshirani and J. Taylor, *An Introduction to Statistical Learning with Applications in Python*. Cham, Switzerland: Springer, 2023.
- [4] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [5] F. Chollet, *Deep Learning with Python*, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2021.
- [6] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [7] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [8] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [9] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [10] J. Schulman, F. Wolski, P. Dhariwal, A. Radford and O. Klimov, "Proximal policy optimization algorithms," arXiv preprint arXiv:1707.06347, 2017.
- [11] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
- [12] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020, pp. 9459–9474.
- [13] J. Johnson, M. Douze and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [14] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.
- [15] F. M. Harper and J. A. Konstan, "The MovieLens datasets: History and context," *ACM Transactions on Interactive Intelligent Systems*, vol. 5, no. 4, Art. 19, 2015.
- [16] Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [17] A. Krizhevsky, "Learning multiple layers of features from tiny images," Technical Report, University of Toronto, 2009.

- [18] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.
- [19] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. 32nd International Conference on Machine Learning (ICML)*, 2015, pp. 448–456.
- [20] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd International Conference on Learning Representations (ICLR)*, 2015.

4.2 Web References, Documentation and Data Sets

- [21] MPOnline Limited, official citizen services portal, <https://www.mponline.gov.in>, and company website, <https://www.mponlinelimited.com>.
- [22] MPOnline Limited, AI/ML Internship Program (AI3): lecture notes, tutorials and assignment briefs (Assignments 1–10), Bhopal, May–July 2026.
- [23] scikit-learn developers, scikit-learn user guide and API reference, <https://scikit-learn.org/stable/>.
- [24] TensorFlow and Keras documentation, <https://www.tensorflow.org> and <https://keras.io>.
- [25] Flask documentation, <https://flask.palletsprojects.com>; Gunicorn documentation, <https://gunicorn.org>.
- [26] Docker documentation, <https://docs.docker.com>.
- [27] Render documentation, <https://render.com/docs>.
- [28] Stable-Baselines3 documentation, <https://stable-baselines3.readthedocs.io>; Gymnasium documentation, <https://gymnasium.farama.org>.
- [29] LangChain documentation, <https://python.langchain.com>; Google Gemini API documentation, <https://ai.google.dev>; Streamlit documentation, <https://docs.streamlit.io>; FastAPI documentation, <https://fastapi.tiangolo.com>.
- [30] Kaggle data sets (<https://www.kaggle.com/datasets/...>): nehalbirla/vehicle-dataset-from-cardexho; mirichoi0218/insurance; blastchar/telco-customer-churn; akram24/position-salaries; uciml/breast-cancer-wisconsin-data; pavansubhasht/ibm-hr-analytics-attrition-dataset; vjchoudhary7/customer-segmentation-tutorial-in-python; oddrationale/mnist-in-csv; bhavikjikadara/dog-and-cat-classification-dataset; johnsmith88/heart-disease-dataset.
- [31] Open-Meteo, Historical Weather API, <https://open-meteo.com>.
- [32] GroupLens Research, MovieLens Latest Datasets (small), <https://grouplens.org/datasets/movielens/>.

ANNEXURE A: PAT COURSE SUMMARY

As required by the department for courses completed through PAT, this annexure gives a short summary of the **Oracle Certified Foundations Associate (Java)** course, which I completed in July 2025, before the internship. The main subject of this report remains the AI/ML internship at MPOnline Limited described in the preceding chapters.

A.1 Course Details

Table A.1 gives the details of the course and the certification.

Table A.1: Details of the PAT course and certification

Item	Details
Certification	Oracle Certified Foundations Associate — Java Certified Foundations Associate
Issued by	Oracle University, Oracle Corporation (Certificate of Recognition)
Date of certification	17th July, 2025
Certificate ID	102022326JAVA8OJA
Undertaken through	PAT, VIT Bhopal University
Course structure	21 modules comprising 65 lessons (about 24 hours) and 38 hands-on practices (about 14 hours)
Platform	Java SE with the Java Development Kit (JDK) and JShell

A.2 Course Content

The course covered Java SE programming from first principles to advanced and enterprise topics. Table A.2 groups its 21 modules into six areas.

Table A.2: Modules of the course grouped by area

Area	Modules	Key topics
Java fundamentals	1 – 2	Java features and object-oriented concepts; compiling and running programs; JDK and JShell; primitive types; arithmetic, bitwise, relational and conditional operators; if/else, switch and the ternary operator
Core Java APIs	3	String and StringBuilder; wrapper classes; BigDecimal; date and time values; localisation and formatting

Area	Modules	Key topics
Object-oriented design	4 – 7	Classes, objects and static context; overloading, constructors, encapsulation and immutability; enumerations; memory allocation and parameter passing; inheritance and polymorphism; sealed, abstract and final classes; records and factory methods; interfaces, generics and composition
Data handling and functional programming	8 – 11	Arrays and loops; Collections API (List, Set, Deque and Map); nested and inner classes; lambda expressions; Streams API, aggregation and parallel streams
Robust applications and I/O	12 – 13	Exception handling, logging, debugging and assertions; Java I/O, standard input and output, serialisation and file systems
Advanced and enterprise topics	14 – 21	Concurrency, executor services and thread safety; modules, packaging and custom runtime images; annotations; JDBC connections and transactions; security best practices; generics erasure; deploying Java applications on Oracle Cloud; design patterns and the file watch service

A.3 Hands-on Practice: Product Management Application

The 38 practices built one Product Management application step by step. Starting from a single Product class, the application gained an enumeration for product ratings, custom constructors and immutable objects; Food and Drink subclasses with overridden methods, factory methods, sealed classes and records; a Rateable interface for product reviews and ratings; a HashMap of products and reviews with sorting and search; nested classes and lambda expressions for customised reports; stream-based processing with a discount calculated from the rating; exception handling and text parsing; reports printed to files, bulk loading of data from files and a memory swap mechanism; a thread-safe singleton ProductManager tested with simulated concurrent callers; and finally conversion of the application into a set of Java modules.

A.4 Learning Outcomes and Relevance to the Internship

Although the course used Java and the internship used Python, many of its ideas carried over directly to the AI/ML work described in this report:

- **Programming to an interface:** because every scikit-learn estimator exposes the same fit() and predict() methods, different models could be swapped freely, for example when five classifiers were compared on the Adult Census data (Section 2.5.7).

- **Exception handling:** the try–except block that returns a clear error message instead of failing in the Car Price web application (Section 2.3.9) and the JSON error response of the heart disease API (Section 2.4.10) follow the same principles.
- **Lambdas, collections and streams:** the functional style of stream pipelines made Pandas operations such as apply() with lambda functions, filtering and grouping natural to write.
- **Serialisation:** saving trained models with Pickle and Joblib is the same idea as Java object serialisation, including the security rule never to deserialise untrusted data.
- **Packaging and deployment:** modules, custom runtime images and cloud deployment prepared the ground for pinning dependencies, building Docker images and deploying services on Render.

A.5 Certificate

The certificate of recognition issued by Oracle University is reproduced in Figure A.1.



Figure A.1: Oracle Certified Foundations Associate certificate (Java Certified Foundations Associate), dated 17th July, 2025